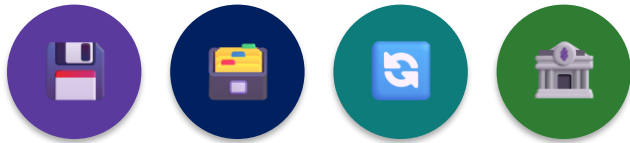


DAY 2

Data and Event-Driven Communication

A two-service workflow using REST validation and TransactionSubmitted events. · Lab 2 — Transaction Service





MODULE 4

Microservices Data and Persistence

Give each service a persistence story: JPA and transactions, database-per-service, saga and outbox patterns, and the banking constraints around consistency and audit.



DATA OWNERSHIP IN MICROSERVICES · DATABASE PER SERVICE PATTERN



BANKING OWNERSHIP EXAMPLE

- Customer Service -> customer profile, contact details
- Account Service -> account records, account status
- Transaction Service -> transaction ledger entries
- Notification Service -> notification preferences and history



WHY OWNERSHIP MATTERS

- Supports independent schema evolution and deployment
- Other services access data through REST APIs, events, or read models -- never direct queries



BAD DESIGN: SHARED DATABASE

- Customer Service, Account Service, Transaction Service, Reporting Service -> shared_bank_db
- Services look separate in code but are tightly coupled through data



BETTER DESIGN: DATABASE PER SERVICE

- Customer Service -> customer_db, Account Service -> account_db, Transaction Service -> transaction_db
- Services communicate through APIs and events rather than shared tables

KEY TAKEAWAY Customer Service owns customer data, Account Service owns account data -- no service reaches into another's tables. • Each service gets its own database or schema -- shared tables would silently recreate the coupling microservices exist to remove.

DAY 2 OBJECTIVES AND CAPSTONE PROGRESS

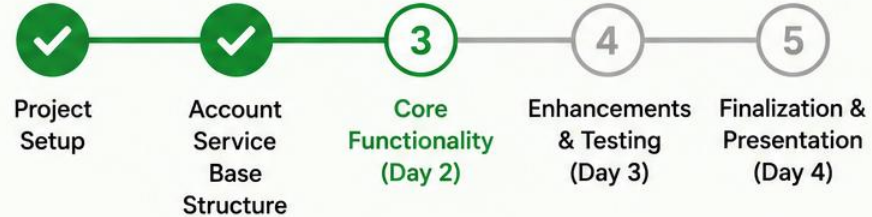


DAY 2 OBJECTIVES

-  Build and integrate Account microservice
-  Implement data persistence with Spring Data JPA
-  Complete REST APIs and test with Postman
-  Implement validation and exception handling
-  Write unit tests and ensure code quality



CAPSTONE PROGRESS



✓	COMPLETED	Project setup, dependencies, and repository initialized
✓	COMPLETED	Account service base structure created, layers and configurations in place
3	IN PROGRESS	Implementing CRUD operations, REST APIs, and data persistence
4	PENDING	Add validations, exception handling, and integration testing
5	PENDING	Finalize code, documentation, and capstone presentation



CAPSTONE GOAL

Deliver a fully functional Account microservice with robust APIs, persistence, validation, testing, and high code quality.

SHARED DATABASE ANTI-PATTERN

A schema change made for one service can silently break every other service reading the same shared tables.

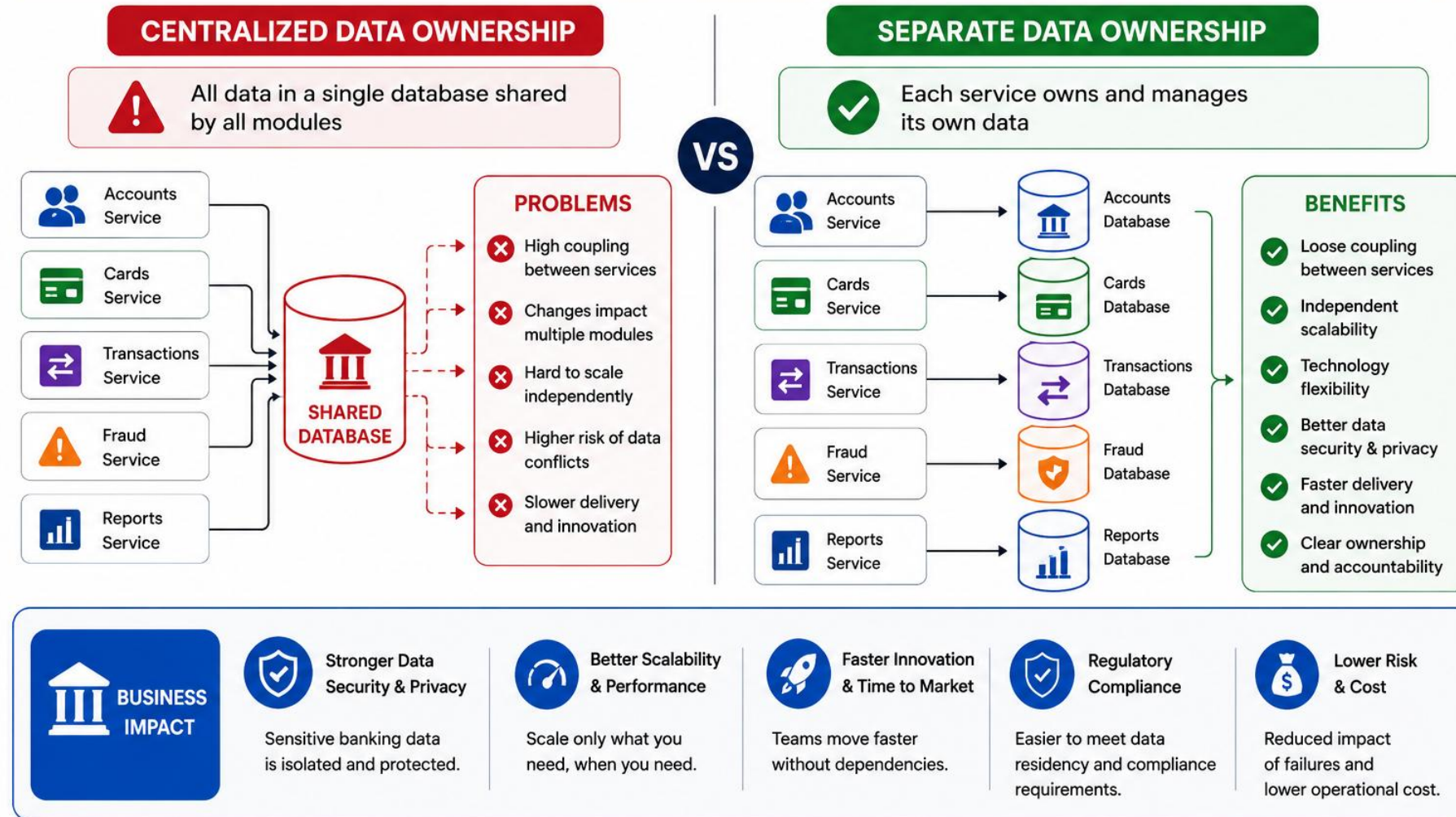


WHY IT'S AN ANTI-PATTERN

- Multiple services become dependent on the same schema, tables, and release timing
- Cross-service queries and joins seem convenient but hide real coupling
- Designing service boundaries around database tables only leads to the same trap

KEY TAKEAWAY A schema change made for one service can silently break every other service reading the same shared tables.

WHY SEPARATE DATA OWNERSHIP MATTERS IN BANKING



POLYGLOT PERSISTENCE

No single database is best for every workload -- Order Service, Payment Service, and Analytics Service can each pick their own fit-for-purpose store.



EXAMPLE DATA STORE CHOICES

- Order Service -> PostgreSQL (relational, ACID, transactions, joins)
- Customer Service -> MongoDB or Cassandra (flexible schema, high write throughput)
- Payment/session data -> Redis (key-value store, fast lookup)
- Analytics Service -> Elasticsearch (full-text search, analytics)



CHALLENGES TO MANAGE

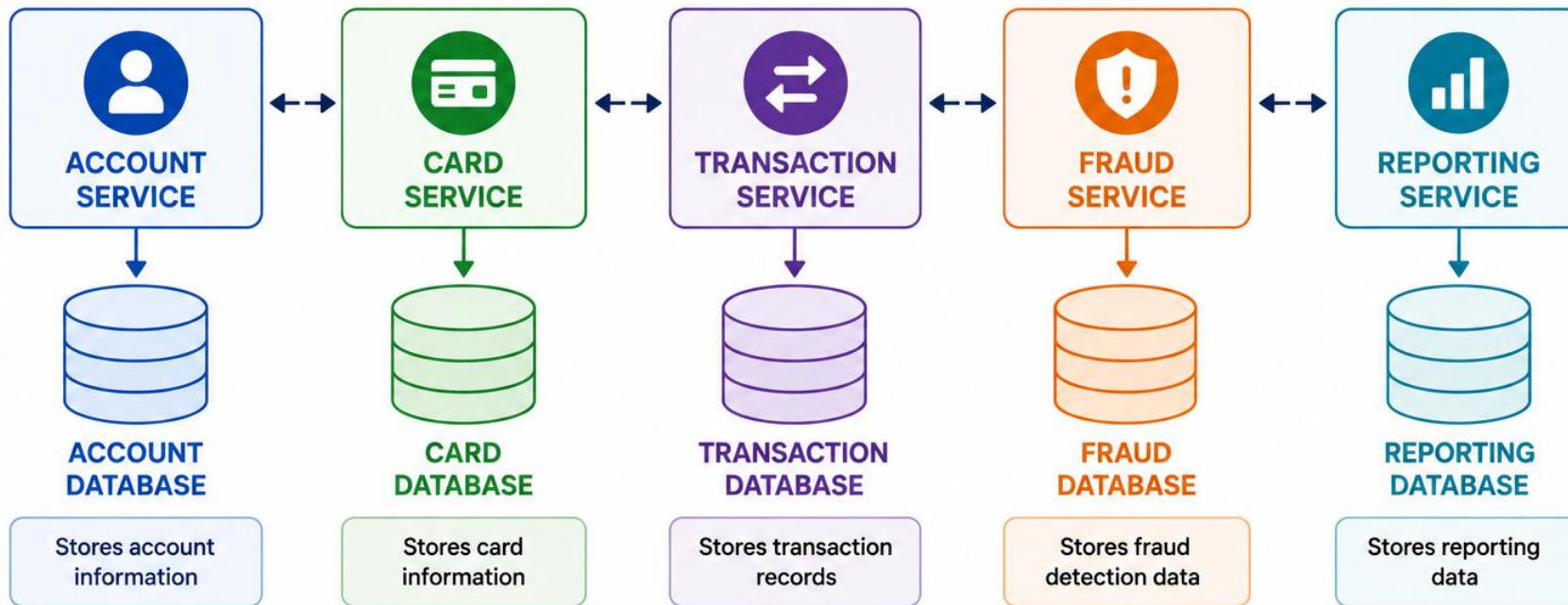
- Increased operational complexity and more skills/tools to manage
- Data consistency across different stores; no joins across databases (use APIs or events)

KEY TAKEAWAY No single database is best for every workload -- Order Service, Payment Service, and Analytics Service can each pick their own fit-for-purpose store.

DATABASE PER SERVICE



Each microservice owns and manages its own database



BENEFITS



Loose coupling between services



Independent scalability



Technology flexibility



Better data security & privacy



Faster delivery and innovation

Relational vs. NoSQL Data Stores



Choosing the right data store depends on your workload, data model, consistency needs, and scalability requirements.

 RELATIONAL DATA STORES (SQL)		 NoSQL DATA STORES (Non-Relational)
 Data Model	Structured data in tables (rows and columns) with a predefined schema.	Flexible data models: key-value, document, wide-column, or graph.
 Relationships	Strong support for relationships using foreign keys and joins.	Limited or no joins; relationships are typically handled within the application.
 ACID Transactions	Full ACID compliance ensures strong consistency and reliability.	Often BASE (Basically Available, Soft state, Eventually consistent) for higher availability.
 Scalability	Primarily vertical scaling (scale up). Horizontal scaling is more complex.	Designed for horizontal scaling (scale out) across many servers.
 Performance	Excellent for complex queries, transactions, and aggregations on structured data.	High performance for large volumes of simple reads/writes and unstructured or semi-structured data.
 Schema	Schema-on-Write: schema is defined upfront and enforced.	Schema-on-Read: schema is flexible and can evolve over time.
 Use Cases	Financial systems, ERP, order management, reporting, data with complex relationships.	Real-time analytics, user profiles, content management, IoT, catalogs, caching, event data.
 Examples	 PostgreSQL  MySQL  Microsoft SQL Server	 mongoDB  cassandra  Redis  elasticsearch



Key Takeaway: Relational databases provide strong consistency and complex query capabilities, while NoSQL databases offer flexibility and scale. Many microservices architectures use **polyglot persistence**—choosing the best data store per service.

SPRING DATA ARCHITECTURE · SPRING DATA JPA FUNDAMENTALS



THE SPRING DATA LAYERS

- Your Spring Boot application -> Spring Data abstraction layer -> store-specific module -> data store
- Store modules: Spring Data JPA, Spring Data MongoDB, Spring Data Cassandra, Spring Data Redis
- Repository hierarchy: Repository -> CrudRepository -> PagingAndSortingRepository -> JpaRepository



JPA FUNDAMENTALS

- Entity: represents a database table, mapped with @Entity
- Primary key: identifies each entity uniquely, mapped with @Id
- Relationships: @OneToMany, @ManyToOne, @OneToOne map associations
- Persistence context: a first-level cache that tracks entity state within a transaction

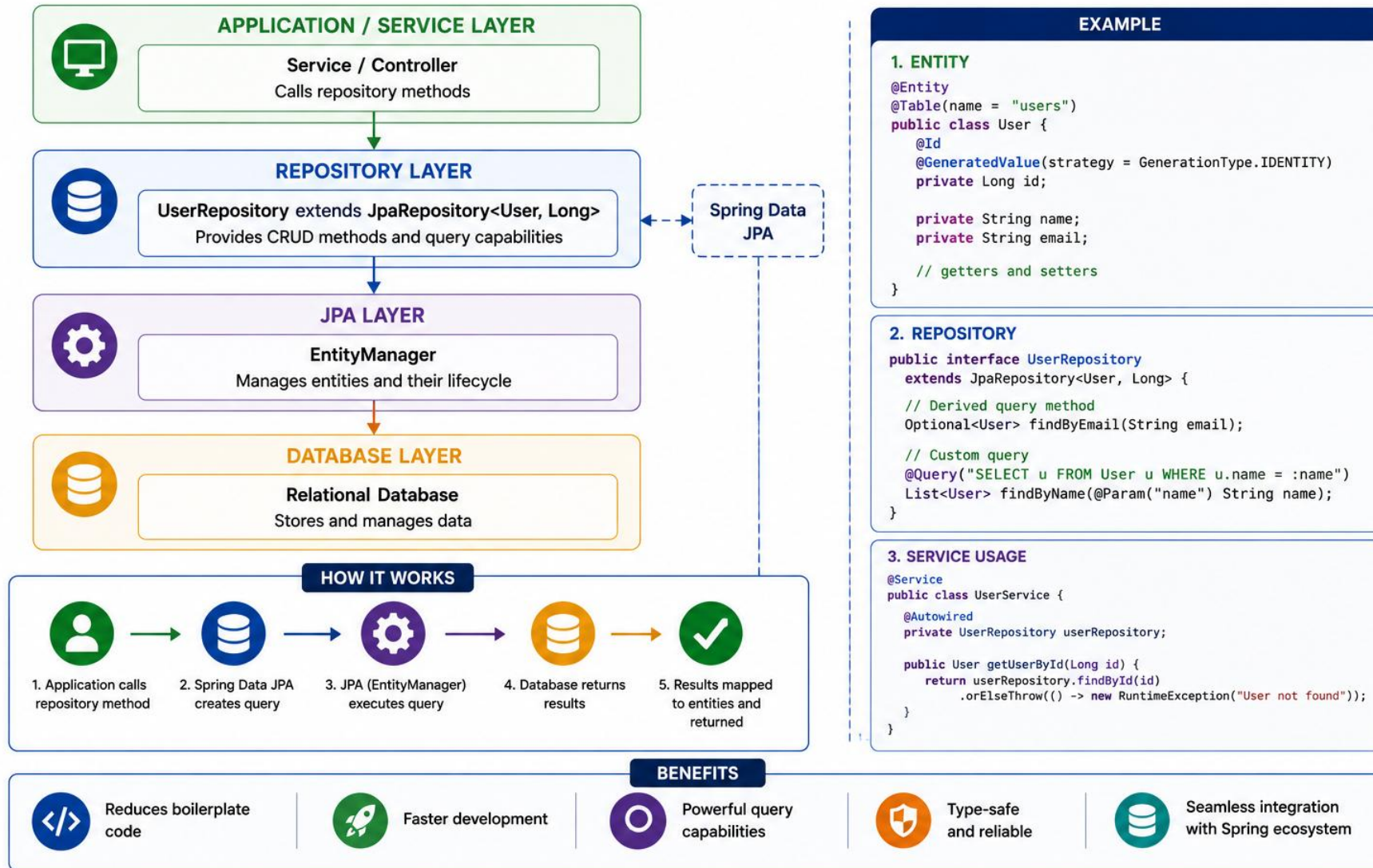


BEST PRACTICES

- Prefer derived queries for simple use cases; use @Transactional on service-layer methods
- Avoid exposing repositories outside the service layer

KEY TAKEAWAY Repository abstraction, template abstraction, domain mapping, and query derivation work the same way whether the store is JPA, MongoDB, or Redis. • Define the entity, extend JpaRepository, and Spring Data JPA generates the SQL -- no boilerplate DAO code required.

SPRING DATA JPA AND REPOSITORY PATTERN



ENTITIES AND OBJECT-RELATIONAL MAPPING

The entity manager, persistence context, dirty checking, and SQL generation together turn changed Java objects into executed SQL statements.



HOW ORM WORKS

1. Entity manager manages the persistence context
2. Persistence context tracks entity states (managed, new, detached, removed)
3. Dirty checking detects changes in entities
4. ORM converts changes to SQL statements and executes them against the database



BEST PRACTICES

- Model the domain, not the database; keep entities small and focused
- Use appropriate fetch types (LAZY by default); keep business logic in the service layer, not the entity

KEY TAKEAWAY The entity manager, persistence context, dirty checking, and SQL generation together turn changed Java objects into executed SQL statements.

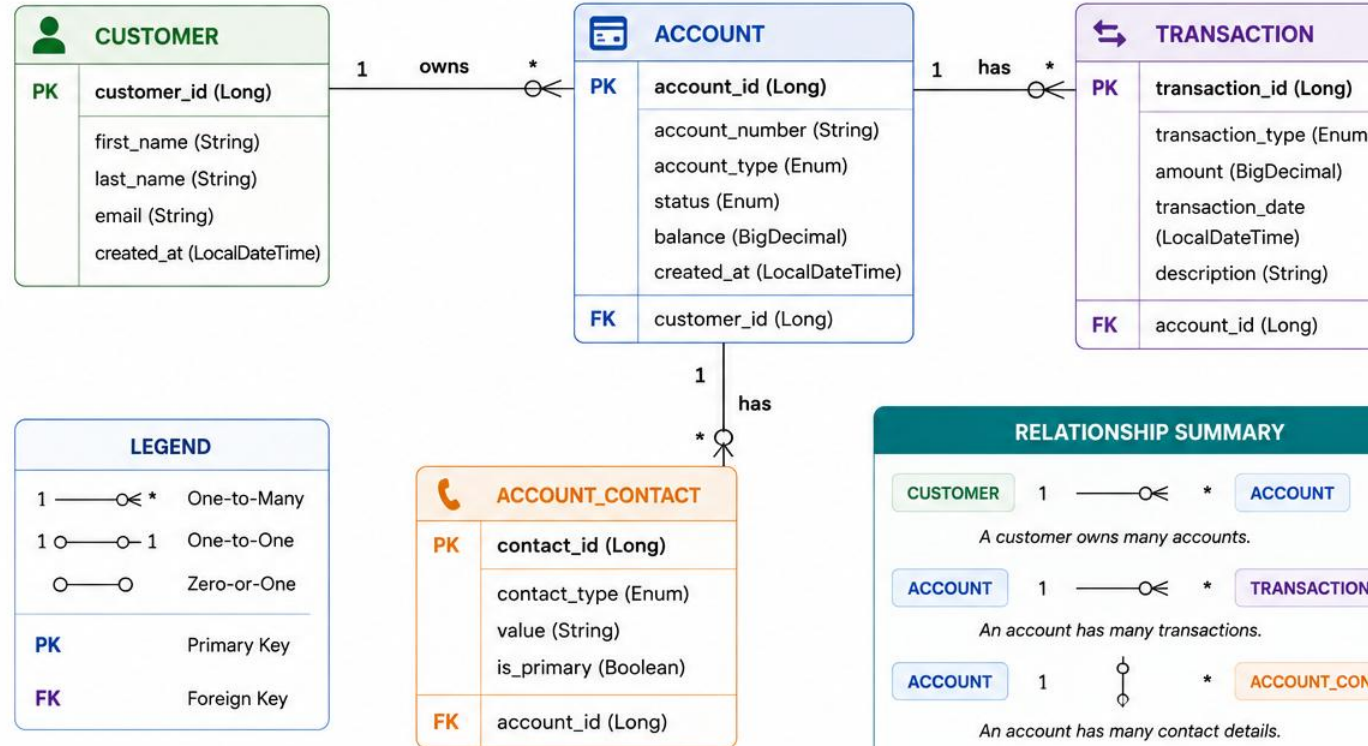
ENTITY DESIGN AND RELATIONSHIPS (WITHIN A SERVICE)



CONTEXT Example: Account Service
Manages customer accounts and related information.

DESIGN PRINCIPLES

- ✓ Data is owned and managed within the service
- ✓ Model based on domain needs
- ✓ Encapsulate data and relationships
- ✓ Use proper keys and constraints
- ✓ Keep model cohesive and consistent



BENEFITS OF GOOD ENTITY DESIGN



Clear domain modeling



Data integrity and consistency



Easier maintenance and evolution



Better performance and scalability



Encapsulated data within the service

REPOSITORY ABSTRACTIONS

CrudRepository, PagingAndSortingRepository, and JpaRepository each add a layer of out-of-the-box CRUD, paging, and JPA-specific capability.

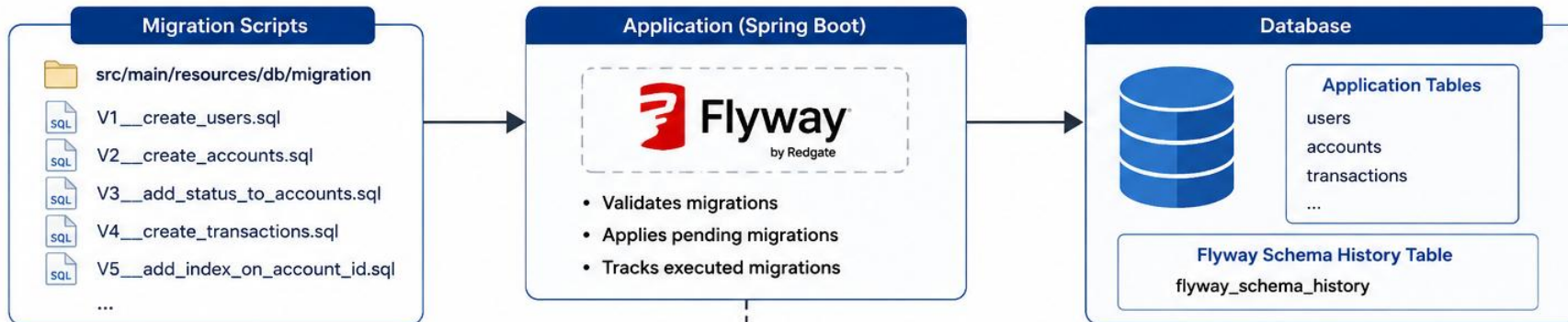
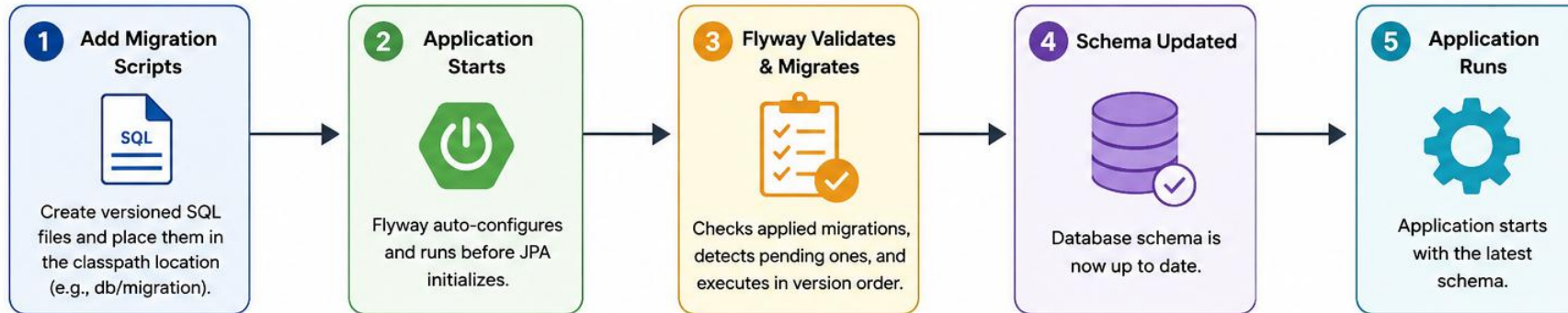


WHY USE REPOSITORY ABSTRACTIONS

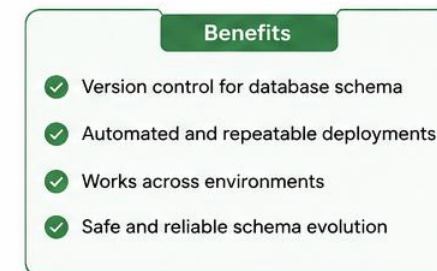
- Decouples business logic from data access implementation
- Consistent programming model across different data stores
- Easier testing with mock repositories; supports automation of common CRUD operations

KEY TAKEAWAY CrudRepository, PagingAndSortingRepository, and JpaRepository each add a layer of out-of-the-box CRUD, paging, and JPA-specific capability.

Flyway Migrations



flyway_schema_history (Tracks Applied Migrations)									
installed_rank	version	description	type	script	checksum	installed_by	installed_on	execution_time	success
1	1	create_users	SQL	V1__create_users.sql	187654321	app_user	2025-05-20 10:15:30	120	true
2	2	create_accounts	SQL	V2__create_accounts.sql	187654322	app_user	2025-05-20 10:15:31	98	true
3	3	add_status_to_accounts	SQL	V3__add_status_to_accounts.sql	187654323	app_user	2025-05-20 10:15:32	75	true
...	...	...	...	...	...	...	...	...	...



ENTITY RELATIONSHIPS AND FETCH STRATEGIES · LAZY LOADING AND THE N+1 PROBLEM



EAGER VS. LAZY

- EAGER: related entity loads immediately in the same query - use only when the association is always needed
- LAZY (default for @OneToMany, @ManyToOne): related entity loads only when accessed



THE PROBLEM

- 1 query to load N parents + N queries (one per parent's lazy collection) = N + 1 queries
- Common in REST APIs returning lists with nested data



COMMON FETCH PITFALLS

- N+1 select problem: accessing LAZY collections in a loop issues one query per parent plus N for children
- Overusing EAGER loads large graphs you don't need, increasing response time and memory usage



HOW TO AVOID IT

- JOIN FETCH: fetch associations in the same query
- @EntityGraph, DTO projections, and batch fetching each solve it a different way
- Always profile real queries (Hibernate SQL logs, p6spy, Datasource proxy)

KEY TAKEAWAY One-to-one, one-to-many, many-to-one, and many-to-many relationships each need a deliberate fetch-strategy decision, not the JPA default alone. • Loading N parent records and then accessing a lazy collection on each one issues 1 + N queries -- and performance degrades as N grows.

QUERY METHODS AND CUSTOM QUERIES · DATABASE CONNECTION POOLING · SCHEMA MIGRATION WITH FLYWAY OR LIQUIBASE



FOUR QUERY APPROACHES

- Derived query methods: Spring Data parses the method name and builds the query automatically
- @Query (custom JPQL): for when the method name would be too long or unreadable
- Native SQL queries: for database-specific features or complex queries
- Named queries: reusable queries defined with @NamedQuery



FLYWAY VS. LIQUIBASE

- Flyway: SQL-first, simple learning curve, manual (SQL) rollback
- Liquibase: SQL, XML, YAML, or Java changesets; more extensible, built-in rollback support



WHY POOLING MATTERS

- Establishing a DB connection is expensive (network handshake, authentication)
- Pooling reuses connections, reducing latency and improving throughput
- Prevents exhausting database connections and improves scalability under load



BEST PRACTICES

- Always version control migration files; make migrations small, atomic, and focused
- Never edit a migration that has already been applied



KEY HIKARICP SETTINGS

- maximum-pool-size, minimum-idle, connection-timeout, idle-timeout, max-lifetime
- Right-size the pool to match database capacity and workload

KEY TAKEAWAY findByEmail, findByStatus, and findByNameContaining are parsed straight from the method name -- no query code required.

- HikariCP, Spring Boot's default pool, cuts latency and prevents connection exhaustion by reusing a right-sized pool of connections.
- Flyway and Liquibase both track applied migrations in a history table so every environment ends up on the exact same schema version.

DATABASE CONFIGURATION ACROSS ENVIRONMENTS · LOCAL TRANSACTIONS WITH SPRING



TYPICAL ENVIRONMENTS

- Development: local or shared DB for developer productivity
- Test/QA: isolated DB for testing and automation
- Staging/UAT: production-like for final validation
- Production: highly available and secured



BEST PRACTICES

- Never commit passwords or URLs; use environment variables or a secrets manager
- Use the least-privilege DB user and validate configuration at startup



HOW A LOCAL TRANSACTION WORKS

- Service layer begins a transaction, executes operations, then commits or rolls back
- Default propagation is REQUIRED; runtime exceptions trigger rollback by default
- Managed by Spring's PlatformTransactionManager, applied via @Transactional

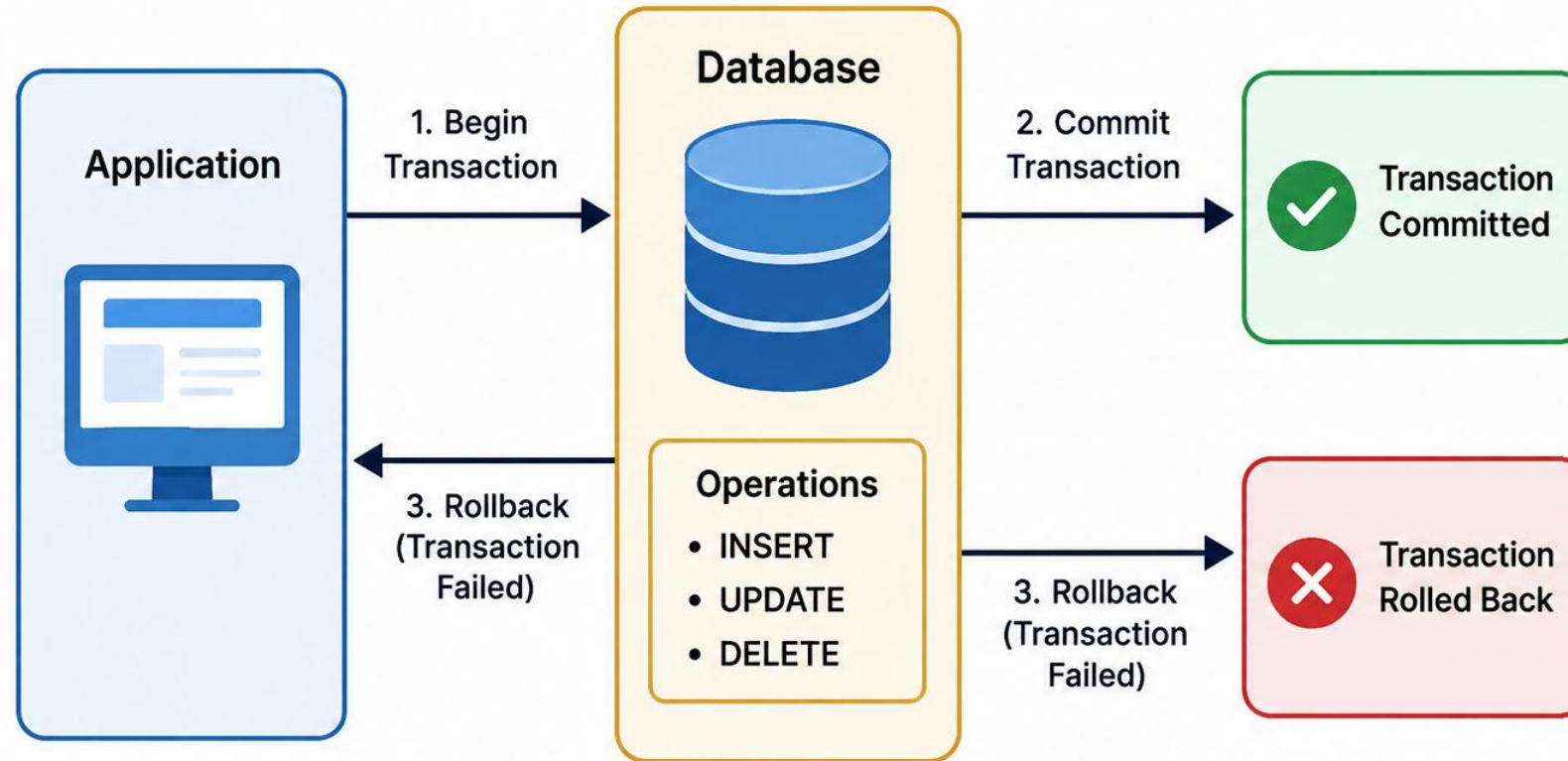


BEST PRACTICES

- Keep transactions short and focused; use @Transactional at the service layer
- Do not perform remote calls (REST, messaging) inside a transaction

KEY TAKEAWAY application-dev.yml, application-qa.yml, and application-prod.yml each point at a different database with different pool sizing and logging. • If any operation inside a @Transactional service method throws, Spring rolls back every change made earlier in that same method.

Local Transactions



TRANSACTION PROPAGATION · TRANSACTION ISOLATION LEVELS

THE FOUR SQL ISOLATION LEVELS

LEVEL	PREVENTS	CONCURRENCY
Read Uncommitted	Nothing	Highest
Read Committed	Dirty reads	High (most systems' default)
Repeatable Read	Dirty + non-repeatable reads	Medium
Serializable	Dirty + non-repeatable + phantom reads	Lowest

KEY PROPAGATION TYPES

- **REQUIRED** (default): join the existing transaction, or create a new one
- **REQUIRES_NEW**: suspend any existing transaction and start an independent one
- **MANDATORY**: must run inside an existing transaction, or throw an exception
- **NEVER**: must not run inside a transaction at all

KEY TAKEAWAY **REQUIRED** is the sensible default for most business operations; **REQUIRES_NEW** isolates audit logging or notifications from the main transaction. • **READ COMMITTED** is the default for most systems and balances consistency against concurrency; **SERIALIZABLE** trades concurrency for full correctness.

OPTIMISTIC AND PESSIMISTIC LOCKING · WHY DISTRIBUTED TRANSACTIONS ARE DIFFICULT



OPTIMISTIC LOCKING

- Assume no conflicts; validate the version at commit time
- @Version field on the entity is checked and incremented automatically by JPA
- High concurrency, but requires retry logic on conflict



WHY IT'S HARD

- No central coordinator knows the status of every participant at all times
- Network latency, partitions, and inevitable service failures make atomicity unreliable
- Locking across services hurts availability, performance, and scalability



PESSIMISTIC LOCKING

- Assume conflicts; lock the row upfront with `@Lock(LockModeType.PESSIMISTIC_WRITE)`
- Prevents conflicts outright, but reduces concurrency and risks deadlocks

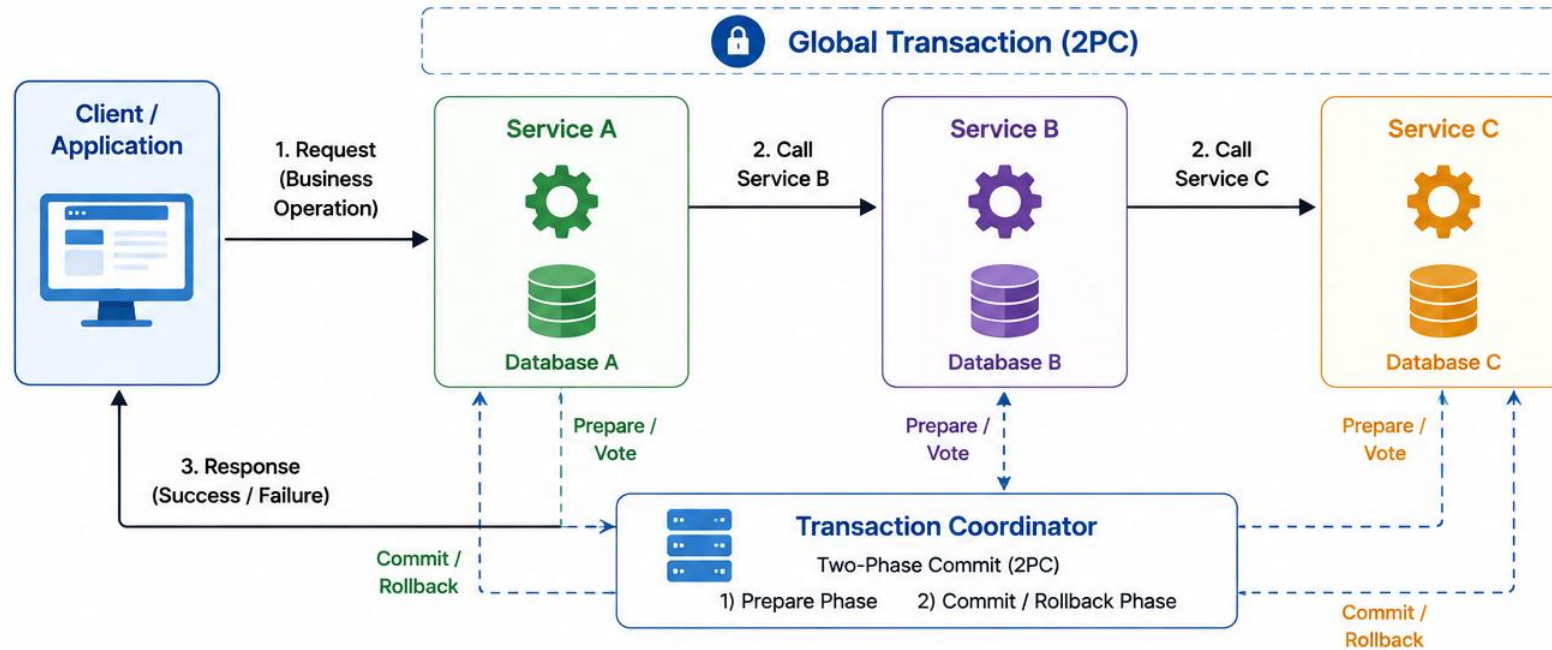


WHAT MICROSERVICES DO INSTEAD

- Avoid distributed transactions where possible
- Use patterns like Saga, Outbox, and eventual consistency; design for compensation, not rollback

KEY TAKEAWAY A @Version field enables optimistic locking for read-heavy, low-conflict data; PESSIMISTIC_WRITE locks suit high-conflict, write-heavy updates. • Once a service commits, it cannot be rolled back automatically if another downstream service fails -- data is left partially updated.

Distributed Transactions - Why Avoid Them



Why Avoid Distributed Transactions



Performance Overhead

Multiple network round trips (prepare & commit) increase latency and reduce throughput.



Blocking & Locks

Resources are locked across services until the transaction completes.



Single Point of Failure

The coordinator is a critical component whose failure can block the entire transaction.



Poor Scalability

Does not scale well with more services, participants, and higher load.



Reduced Availability

If any participant is slow or down, the whole transaction can fail or time out.



Operational Complexity

Hard to implement, test, monitor, and debug across multiple services.

Better Alternatives



Local Transactions

Keep ACID within each service.



Event-Driven Architecture

Use events to coordinate between services.



Sagas (Choreography / Orchestration)

Break long transactions into a sequence of local transactions with compensation.



Idempotency & Retries

Ensure reliable processing without distributed locks.



Outbox Pattern

Publish events reliably from local transactions.

TWO-PHASE COMMIT AND ITS LIMITATIONS · EVENTUAL CONSISTENCY · SAGA PATTERN OVERVIEW



HOW 2PC WORKS

- Phase 1 (Prepare): coordinator asks all participants if they are ready to commit
- Phase 2 (Commit/Rollback): if all voted yes, coordinator tells everyone to commit; otherwise, rollback



LIMITATIONS

- Blocking protocol: a coordinator failure after Phase 1 leaves participants stuck
- High latency, poor scalability, and not resilient to network partitions



HOW IT WORKS

- A write goes to the primary; the update propagates asynchronously to replicas
- For a window of time, different reads may return different values
- After propagation completes, all nodes converge to the same value



KEY CHARACTERISTICS

- No immediate consistency, but guaranteed convergence and tolerance for failures
- Trade-off: sacrifices immediate consistency for availability and scalability



HOW A SAGA WORKS

- Each service completes its own local transaction, then triggers the next step
- If a step fails, previously completed steps are compensated in reverse order
- Can be implemented via orchestration (central coordinator) or choreography (event-driven)



BENEFITS

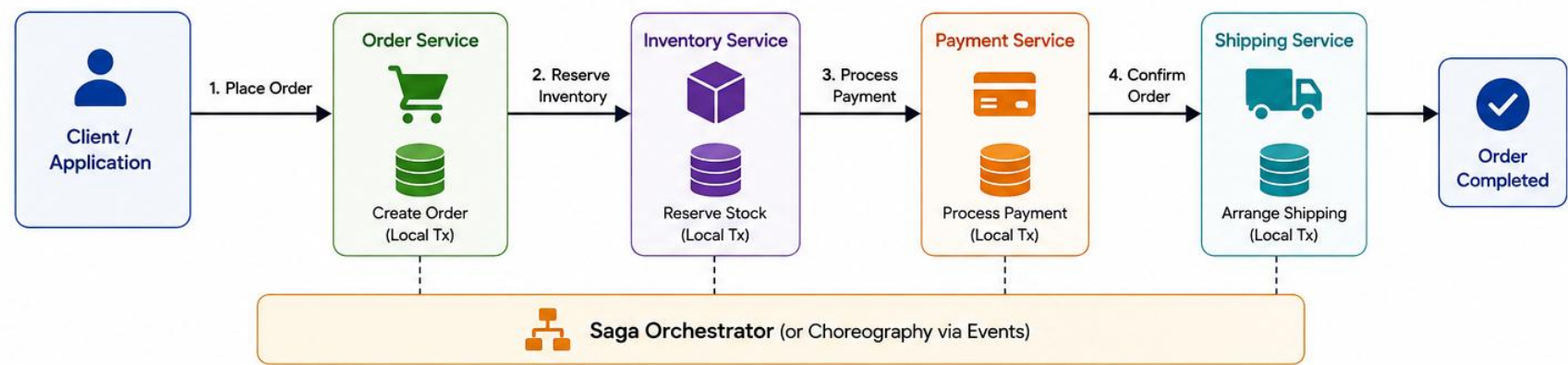
- No distributed locks or 2PC; better scalability, availability, and fault isolation

KEY TAKEAWAY If the coordinator fails after the prepare phase, every participant blocks indefinitely waiting for a decision that may never come.

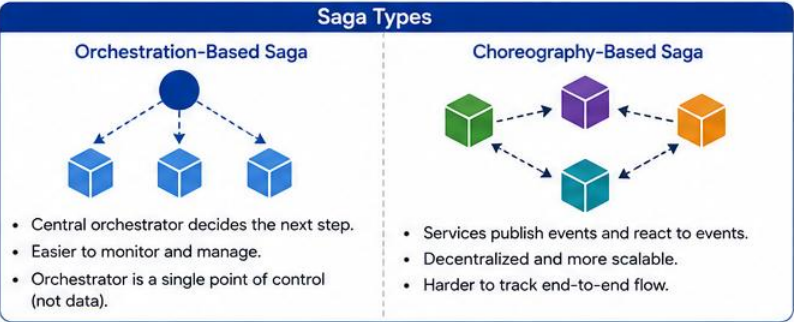
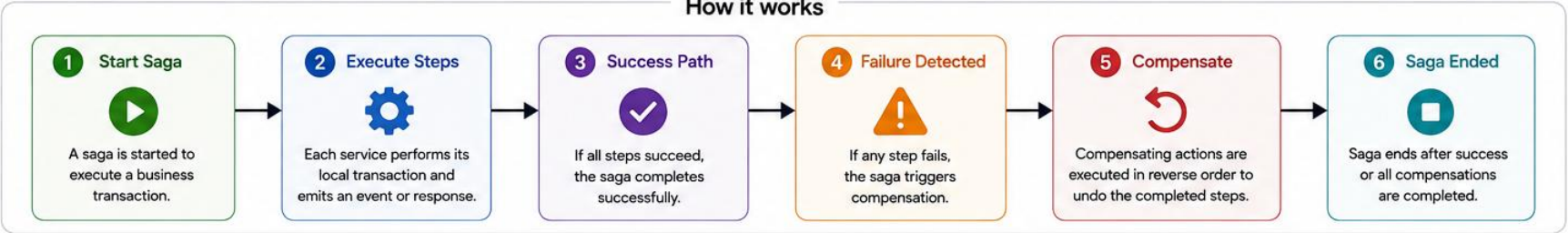
- Different nodes may briefly disagree after a write, but the system guarantees they converge to the same state once updates stop arriving.
- If step three of a saga fails, steps one and two are undone in reverse order through their own compensating actions -- not a database rollback.

Saga Pattern Overview

Manage distributed transactions as a sequence of local transactions with compensating actions.



How it works



Key Characteristics	
Local Transactions	Each step updates its own database.
Event-Driven	Communication via events or messages.
Eventual Consistency	System reaches a consistent state over time.
Compensating Actions	Used to undo previously completed steps on failure.
No Locks	Avoids long-running locks across services.

CHOREOGRAPHY-BASED SAGAS · ORCHESTRATION-BASED SAGAS

· COMPENSATING TRANSACTIONS



HOW CHOREOGRAPHY WORKS

- A service performs a local transaction, then publishes an event describing what happened
- Other services listen for that event and perform their own local transaction and next event



BENEFITS AND CHALLENGES

- Decentralized, highly scalable, and loosely coupled -- no single point of failure
- Harder to visualize the full business flow and to trace/debug across many services



HOW ORCHESTRATION WORKS

- Client requests a business operation; orchestrator starts a saga instance
- Orchestrator sends a command to each participant, records progress, and proceeds
- On failure, the orchestrator triggers compensating transactions in reverse order



ADVANTAGES AND CONSIDERATIONS

- Centralized control, visibility, and easier error handling
- The orchestrator becomes a critical component and a potential bottleneck



EXAMPLES OF COMPENSATION

- Cancel an order, release reserved inventory, refund a payment
- Executed only when a failure occurs, always in reverse order of the completed steps



CONSIDERATIONS

- Compensation actions must be carefully designed and idempotent
- Not always possible to perfectly undo an action -- the business must tolerate temporary inconsistency

KEY TAKEAWAY AccountOpened triggers Transaction Service to record the deposit, which in turn can trigger Notification Service -- all without central control. • AccountOpeningOrchestrator sends commands to Customer, Account, and Notification services and triggers compensations if any step fails. • Closing an account that was opened but never funded, or reversing a transaction, brings the system to an acceptable state, not a rewritten history.

DATA CONSISTENCY TRADEOFFS · CQRS PATTERN OVERVIEW

CONSISTENCY REQUIREMENTS BY OPERATION

OPERATION	CONSISTENCY NEED
Deduct funds from account	Strong consistency inside the owning service
Send notification	Eventual consistency acceptable
Update reporting dashboard	Eventual consistency acceptable
Validate login	Immediate response required

COMMAND VS. QUERY RESPONSIBILITY SEGREGATION

- Write side: commands validate business rules and update the source-of-truth data model
- Read side: queries run against a denormalized, purpose-built read model
- The read model is updated asynchronously, typically by consuming domain events

WHEN CQRS HELPS

- Reporting and dashboard use cases that would otherwise require expensive cross-service joins
- Read and write workloads that need to scale independently

KEY TAKEAWAY Deducting funds needs strong consistency; updating a reporting dashboard can tolerate eventual consistency -- the business decides which. • Commands update the owning service's write model; queries run against a purpose-built read model kept in sync through events.

API COMPOSITION FOR DISTRIBUTED DATA · DATA PRIVACY AND SERVICE BOUNDARIES · DATABASE PERFORMANCE CONSIDERATIONS



HOW IT WORKS

- A composition layer calls each owning service for its slice of the data
- Results are merged in application code to build one client-facing response
- Useful for read-heavy views that combine data owned by multiple services



COMMON PERFORMANCE ISSUES

- Missing indexes, inefficient queries, and N+1 query problems
- Loading too much data, large response payloads, poor connection pool settings



PRIVACY-AWARE DATA DESIGN

- The owning service controls exactly what fields it exposes through its API
- Downstream services store only the identifiers they need, not full customer records
- Events should avoid carrying sensitive fields unless explicitly required and protected



BEST PRACTICES

- Add indexes for common lookup fields (customer email, account number, transaction date)
- Measure before optimizing; use pagination and DTO projections for large or summary result sets

KEY TAKEAWAY A composer calls Customer, Account, and Transaction services in parallel and joins their responses in application code, not in SQL. • Only Customer Service should hold full PII; other services should reference a customerId and request only what they specifically need. • Missing indexes and N+1 queries are the most common root causes of a slow banking API -- and both are fixable before caching is even considered.

DATA PERSISTENCE ANTI-PATTERNS · CACHING FUNDAMENTALS · WHEN CACHING HELPS-AND WHEN IT DOES NOT

ANTI-PATTERNS TO AVOID

- Returning JPA entities directly from APIs instead of DTOs
- Sharing a database across services; using ddl-auto: update in production
- Ignoring lazy loading behavior and eager-loading everywhere
- Creating extremely large, deeply nested entity graphs

GOOD CANDIDATES

- Frequently read data that is expensive to compute or changes infrequently
- Reference data and aggregated read models

WHAT A CACHE STRATEGY MUST DEFINE

- What data is cached and where it is cached (local vs. distributed)
- How long it lives (TTL) and when it is invalidated
- What happens to the application when the cache is unavailable

POOR CANDIDATES

- Highly sensitive data, rapidly changing balances
- Data requiring immediate accuracy, large unbounded result sets

KEY TAKEAWAY Returning JPA entities directly, sharing databases, and skipping migrations are small shortcuts with outsized long-term cost. • What to cache, where, for how long, and how to invalidate it are all design decisions, not defaults to accept blindly. • Account balances and fraud scores are poor caching candidates; reference data and product configuration are excellent ones.

CACHE-ASIDE PATTERN · READ-THROUGH AND WRITE-THROUGH CACHING · WRITE-BEHIND CACHING

CACHE-ASIDE FLOW

1. Application checks the cache for the requested key
2. If found, return the cached value directly
3. If not found, query the database, store the result in the cache, then return it

WRITE-THROUGH

- Writes go to the cache first, which synchronously writes through to the database
- Keeps cache and database aligned, at the cost of added write latency

READ-THROUGH

- On a cache miss, the cache itself loads the value from the database and stores it
- Application code always talks to the cache, never directly to the database for reads

HOW WRITE-BEHIND WORKS

- Write goes to the cache, which returns immediately
- The cache asynchronously flushes the write to the database, often batched
- Improves write latency and throughput, but risks data loss if the cache fails before flushing

KEY TAKEAWAY @Cacheable implements cache-aside out of the box -- the application, not the cache, owns the responsibility of keeping data fresh. • Instead of the app deciding when to populate the cache, the caching layer itself loads on a miss and writes through on every update. • Fast writes come at the cost of a short window where cache and database can disagree if the service fails before the write flushes.

REDIS ARCHITECTURE AND DATA STRUCTURES · SPRING CACHE ABSTRACTION · INTEGRATING REDIS WITH SPRING BOOT

REDIS DATA STRUCTURES

- Strings and hashes: common for basic caching of values and objects
- Lists, sets, and sorted sets: queues, uniqueness checks, and leaderboards
- Streams: lightweight event/message logs

TYPICAL SETUP

- spring.data.redis.host / port configuration in application.yml
- @EnableCaching on the application class activates Spring's cache abstraction
- @Cacheable(value = "customers", key = "#id") caches a repository lookup method

CORE CACHING ANNOTATIONS

- @Cacheable: cache the method's return value, keyed by its arguments
- @CachePut: always run the method and update the cache with the result
- @CacheEvict: remove an entry (or clear a cache) when data changes

IMPORTANT CAUTION

- Redis should not casually become the system of record for critical banking data
- The relational database remains the source of truth; Redis acts as a cache

KEY TAKEAWAY Strings, hashes, lists, sets, sorted sets, and streams give Redis far more flexibility than a plain cache -- rate limiting, queues, and counters included. • @Cacheable, @CachePut, and @CacheEvict express caching intent declaratively -- the actual cache backend is a configuration decision. • spring.data.redis.host/port configuration plus @Cacheable(value="customers", key="#id") is enough to cache a lookup method.

CACHE KEYS AND VALUE DESIGN · CACHE EXPIRATION AND TIME-TO-LIVE · CACHE INVALIDATION STRATEGIES



CACHE KEY DESIGN PRINCIPLES

- Namespace keys by service and entity type, e.g. customer:101:profile
- Keep values small and focused -- cache DTOs or summaries, not full entity graphs
- Use consistent, meaningful key patterns so keys can be found, scanned, or bulk-invalidated



MATCHING TTL TO DATA

- Account status: short TTL
- Customer profile: moderate TTL
- Product configuration: longer TTL
- Fraud risk score: very short TTL

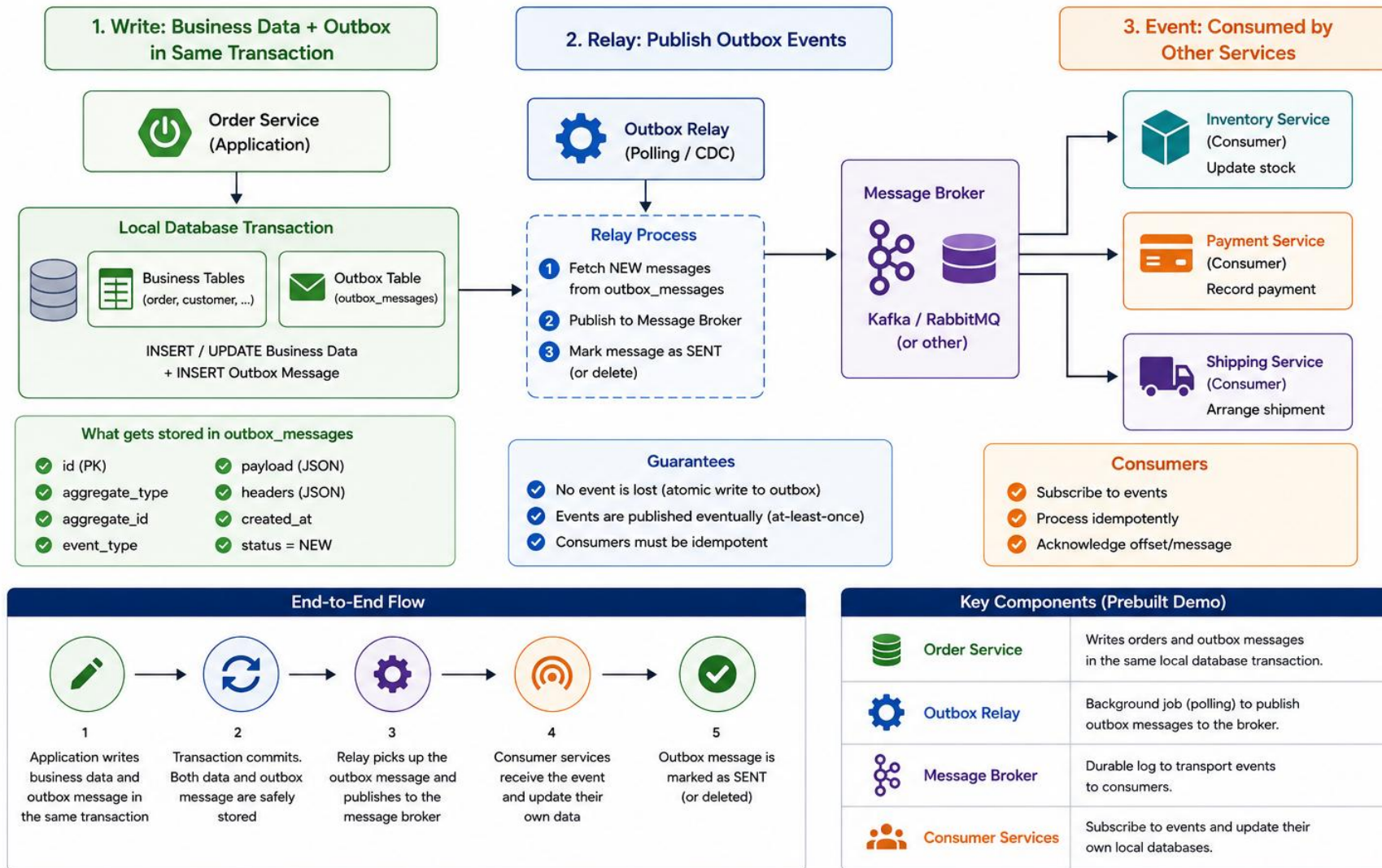


INVALIDATION TRIGGERS

- Time-to-live expired
- Underlying data was updated (evict via @CacheEvict on the write path)
- Memory limit reached, or manual invalidation was triggered

KEY TAKEAWAY customer:101:profile beats a bare 101 -- namespacing prevents key collisions and makes bulk invalidation by prefix possible. • Account status needs a short TTL, fraud risk score needs a very short TTL, and product configuration can tolerate a much longer one. • When a customer updates their phone number, the write must update the database and evict or refresh the cached profile in the same operation.

Transactional Outbox Overview (prebuilt demo)



PREVENTING STALE DATA · CACHE STAMPEDE AND PENETRATION



PREVENTING STALE DATA

- Evict or update the cache in the same transaction or operation as the database write
- Use short TTLs for data where staleness has real business consequences
- Never cache highly volatile data (account balances) without an explicit invalidation path



TWO RELATED FAILURE MODES

- Cache stampede: a hot key expires and many concurrent requests all recompute it at once
- Cache penetration: repeated requests for a nonexistent key never get cached, always hitting the database

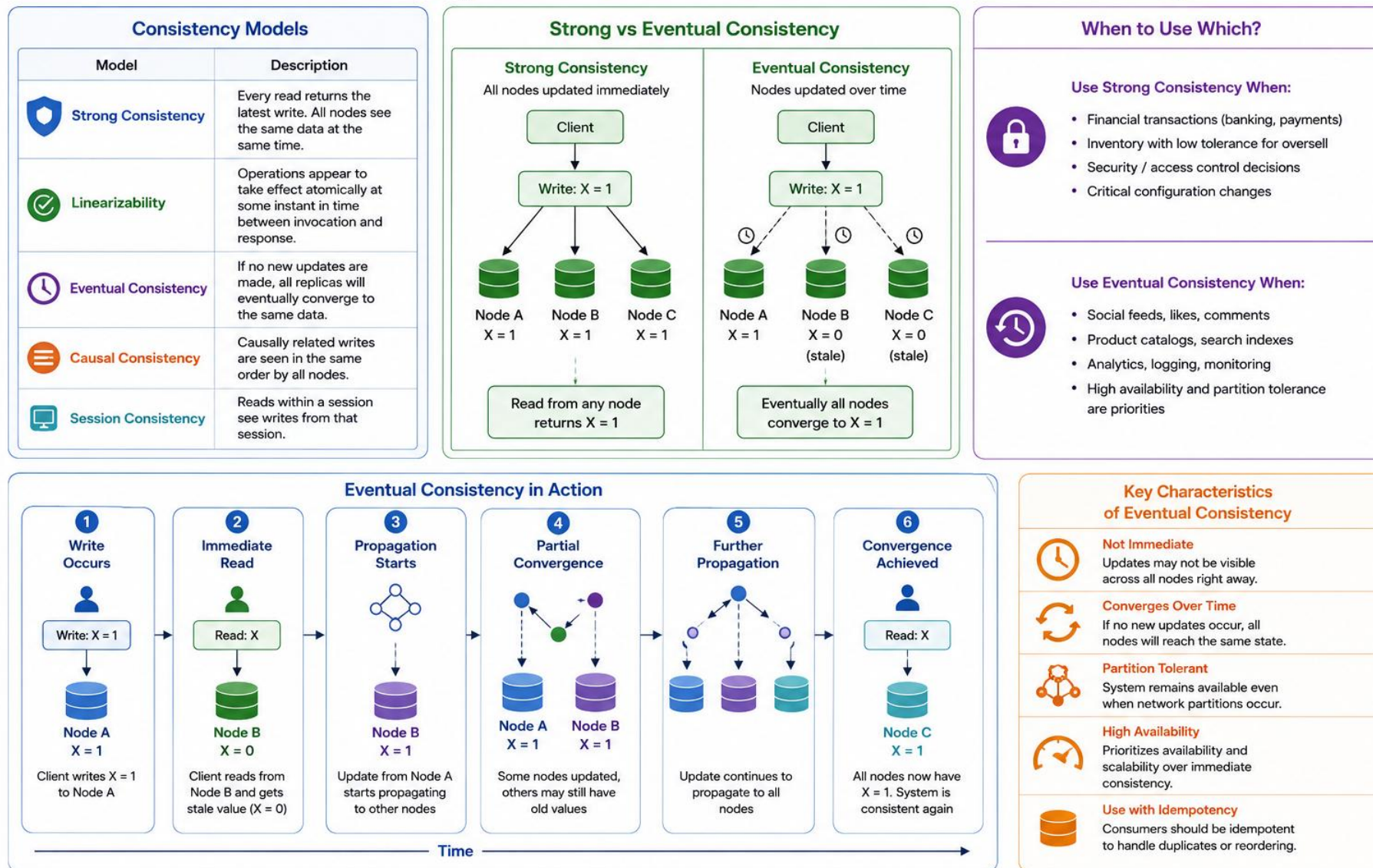


MITIGATIONS

- Locking or request coalescing on cache-miss recomputation
- Cache negative results briefly, or use a bloom filter to short-circuit lookups for known-missing keys

KEY TAKEAWAY Pairing every cache-populating read with a cache-evicting write is the single most important discipline in cache design. • Cache penetration is the related risk of repeated lookups for keys that don't exist at all, bypassing the cache on every single request.

Consistency Models and Eventual Consistency



DISTRIBUTED CACHING CONSIDERATIONS · REDIS HIGH AVAILABILITY · SECURING REDIS



KEY CONSIDERATIONS

- Partitioning/sharding strategy across the cluster as data volume grows
- Consistent key design across every service instance sharing the cache
- Defined fallback behavior for the application when the cache cluster is unavailable



HIGH AVAILABILITY OPTIONS

- Primary-replica replication for read scaling and failover
- Redis Sentinel for automatic failover and monitoring
- Redis Cluster for sharded, horizontally scaled deployments



SECURING REDIS

- Require authentication (requirepass or Redis ACLs) -- never run open
- Restrict network access to only the services that need it
- Enable TLS for data in transit, and avoid caching sensitive data unless explicitly protected

KEY TAKEAWAY Multiple service instances sharing one Redis cluster must agree on key design, eviction policy, and what happens if the cluster is unreachable.

- A single Redis instance is a single point of failure -- replication with automatic failover is what makes caching production-safe.
- Authentication, network isolation, and TLS are not optional extras for Redis in a banking platform -- they're baseline requirements.

Monitoring Cache Performance

Measure. Observe. Optimize.



Effective monitoring ensures your cache delivers **speed**, **reliability**, and **cost efficiency**.

WHY MONITOR CACHE?



Ensure Performance

Track hit rates and latency to ensure expected speed.



Ensure Reliability

Detect issues early and prevent outages.



Optimize Cost

Right-size memory and avoid over-provisioning.



Capacity Planning

Understand usage trends to plan for growth.

BEST PRACTICES

- ✓ Define and monitor SLOs for cache metrics.
- ✓ Alert on anomalies and threshold breaches.
- ✓ Correlate cache metrics with application performance.
- ✓ Review eviction causes and memory fragmentation.
- ✓ Regularly analyze key patterns and TTL usage.



Remember:

A cache is fast only when it hits, healthy when it's stable, and valuable when it's right-sized.



Tools:

Redis INFO, Prometheus, Grafana, Alertmanager, OpenTelemetry, and Application Metrics



KEY CACHE METRICS

Hit Rate



% of requests served from cache. Higher is better.

Target: > 80%

Miss Rate



% of requests not in cache.

Target: < 20%

Latency



Time to get a response from cache (p50, p95, p99).

Target: Low & Stable

Evictions



Keys removed due to memory pressure.

Target: Low

Memory Usage



Used memory vs. total allocated memory.

Target: < 85%

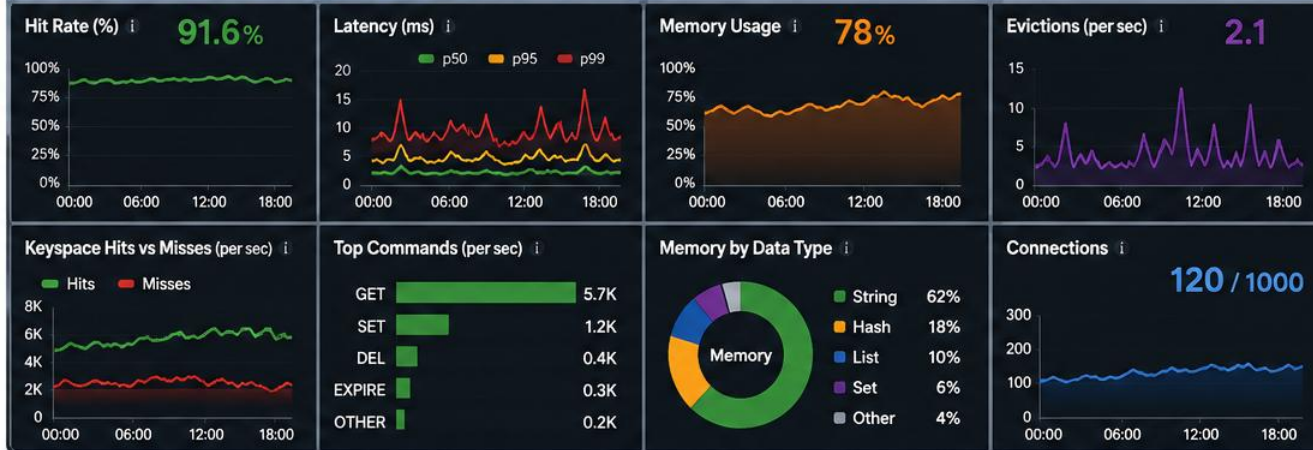
Throughput



Commands per second (reads + writes).

Monitor Trends

DASHBOARD EXAMPLE (Redis + Prometheus + Grafana)



MODULE 5

Integration and Event-Driven Messaging

Connect services without a distributed monolith: Kafka and Spring events, idempotent consumers, outbox, and the failure modes that show up in a payment flow.



COMMUNICATION CHALLENGES IN DISTRIBUTED SYSTEMS

Latency, service unavailability, duplicate messages, and versioning turn simple service-to-service calls into real engineering problems.



COMMON INTEGRATION CHALLENGES

- Network latency, service unavailability, and timeout handling
- Duplicate messages, data consistency, and API versioning
- Contract changes, error handling, and security between services



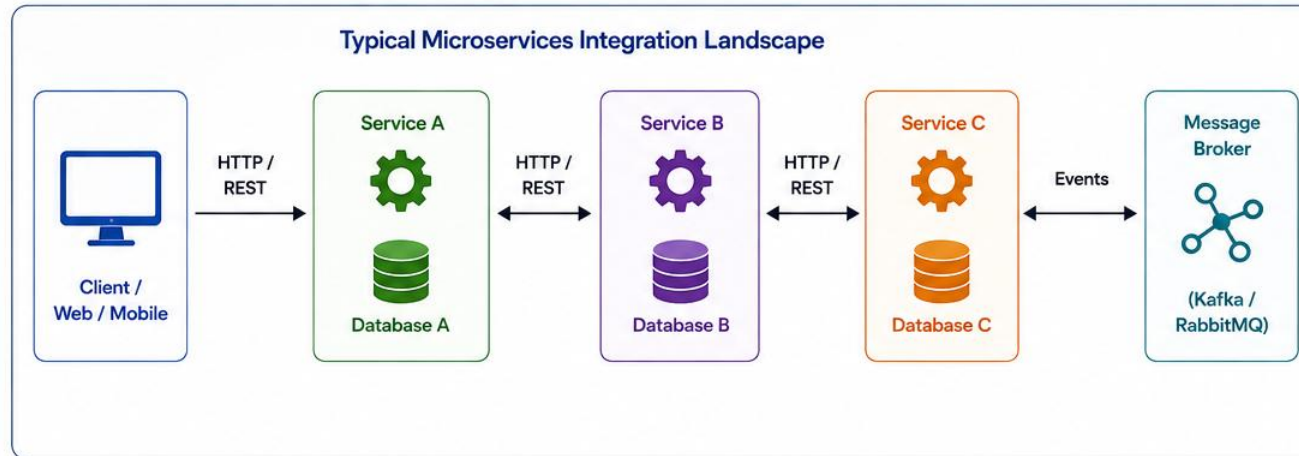
BANKING EXAMPLE

- If Account Service is unavailable, Transaction Service must choose: reject, retry, queue as pending, use cached data, or publish an event for later
- The correct answer depends on the business requirement, not the technology

KEY TAKEAWAY Latency, service unavailability, duplicate messages, and versioning turn simple service-to-service calls into real engineering problems.

Integration Challenges

Building distributed systems means integrating independent services—introducing complexity beyond a monolith.



Why Integration Becomes Harder



Multiple Services

More services = more dependencies and integration points.



Network Calls

Unreliable networks introduce latency, timeouts, and failures.



Independent Data

Data is distributed; no single source of truth.



Asynchronous Nature

Events and eventual consistency make coordination difficult.



Partial Failures

Some services succeed while others fail—hard to handle consistently.

Key Integration Challenges



1. Network Latency and Timeouts

- Network calls are slower than in-process calls.
- Timeouts can cause failures even when services are healthy.



2. Service Unreliability

- Services can crash, restart, or become unavailable.
- Integration must handle retries and failover.



3. Data Consistency Issues

- No shared database.
- Maintaining consistency across services is hard.
- Leads to eventual consistency.



4. Transaction Management

- Distributed transactions are complex and costly.
- Requires patterns like Saga and Outbox.



5. Versioning and Compatibility

- Services evolve independently.
- Backward/forward compatibility must be maintained.



6. Security and Governance

- Secure communication between services.
- Authentication, authorization, and auditing are essential.

How We Address These Challenges



Resilience Patterns

Retries, timeouts, circuit breakers, bulkheads.



Asynchronous Communication

Use events and messaging to decouple services.



Data Ownership

Each service owns its data; design for eventual consistency.



Saga Pattern

Manage distributed transactions with compensating actions.



API Contracts & Versioning

Define clear contracts and version APIs.



Observability

Centralized logging, metrics, tracing, and alerting.

SERVICE-TO-SERVICE COMMUNICATION OPTIONS ·

SYNCHRONOUS COMMUNICATION WITH HTTP



TWO BROAD INTEGRATION STYLES

- Synchronous communication: commonly implemented with REST APIs, caller waits for a response
- Asynchronous communication: commonly implemented with messaging, events, and brokers
- Most real banking workflows use a deliberate mix of both



REST API CALL FLOW

- Client -> Transaction Service -> Account Service -> Account Database -> response flows back
- Easy to understand, but creates a runtime dependency between the two services



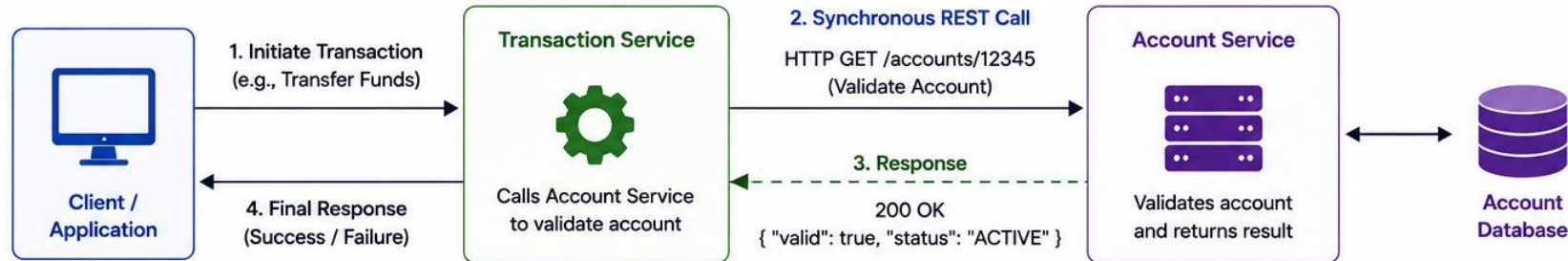
BEST PRACTICES

- Set connection and read timeouts on every synchronous call
- Avoid long chains of synchronous calls; retry only safe (idempotent) operations

KEY TAKEAWAY Every integration decision starts with one question: does the caller need an answer right now, or can the work happen later? • Transaction Service calling GET /accounts/5001 is simple and testable, but Account Service must be healthy for the call to succeed.

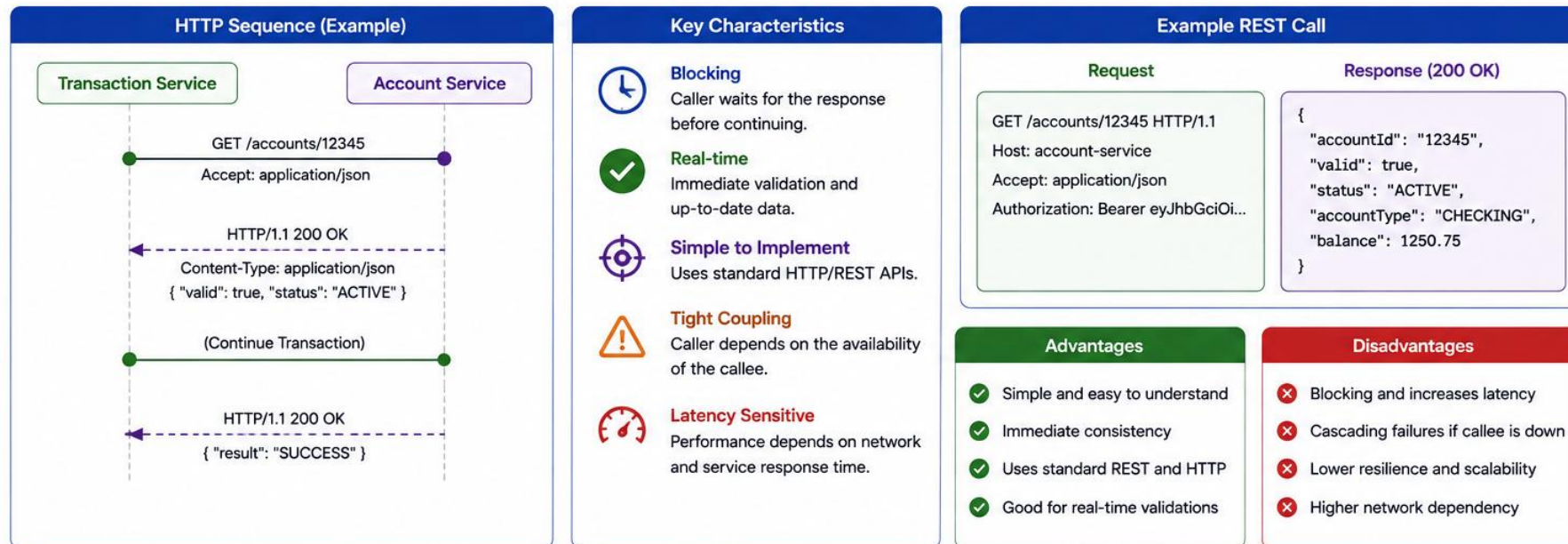
Synchronous Communication (REST Account Validation)

A service makes a blocking REST call to another service and waits for the response.



Synchronous = Blocking

The Transaction Service waits for the Account Service response.
If the Account Service is slow or unavailable, the transaction is delayed or fails.



DECLARATIVE HTTP CLIENTS · SPRING RESTCLIENT AND WEBCLIENT · BLOCKING VS. NON-BLOCKING COMMUNICATION



WHY DECLARATIVE CLIENTS HELP

- Define the remote service contract as a Java interface with annotations
- The framework generates the HTTP call implementation automatically
- Reduces boilerplate compared to manually building requests for every call



RESTCLIENT: THE MODERN SYNCHRONOUS CLIENT

- Blocking, imperative API built on Spring's `HttpService` and `HttpClient`
- Ideal for traditional MVC applications and simpler use cases



WEBCLIENT: THE REACTIVE, NON-BLOCKING CLIENT

- Non-blocking, reactive API built on Project Reactor (Mono/Flux) and Netty
- Ideal for reactive applications, high concurrency, and streaming APIs (SSE)



BLOCKING COMMUNICATION

- The calling thread waits until the response is received
- Simple, imperative model; consumes one thread per request during the wait
- Good fit for traditional MVC apps and low-to-moderate traffic



NON-BLOCKING COMMUNICATION

- The calling thread is not blocked and can do other work while waiting
- Uses fewer threads, enabling much higher concurrency for I/O-bound workloads

KEY TAKEAWAY Annotate an interface with the target endpoints and the client framework generates the HTTP call implementation for you. • `RestClient.builder().get().uri(...).retrieve().body(...)` and `WebClient's Mono`-returning equivalent share a similar fluent API for different threading models. • A blocking call holds one thread per request until the response arrives; a non-blocking call frees the thread to do other work while it waits.

REQUEST TIMEOUTS AND CONNECTION MANAGEMENT · SERVICE DISCOVERY AND CLIENT-SIDE LOAD BALANCING



TIMEOUTS TO CONFIGURE

- Connection timeout: max time to establish a connection to the server
- Connection request timeout: max time to wait for a connection from the pool
- Read/response timeout: max time to wait for data after the request is sent



CONNECTION POOLING ESSENTIALS

- Reuse connections instead of creating new ones -- reduces latency and resource usage
- Tune pool size based on concurrency and downstream capacity; evict idle connections



HOW IT WORKS

1. Client queries the service registry for available instances of a service
2. Registry returns a healthy instance list with metadata
3. Client's load balancer selects an instance (round robin, random, least connections, weighted)
4. Client sends the request to the selected instance and retries elsewhere if it's unhealthy

KEY TAKEAWAY Connection, connection-request, read, and write timeouts each guard a different phase of an HTTP call from waiting forever. • A service registry (Eureka, Consul, etcd) tracks live instances; the client's load balancer then picks a healthy one for each request.

API GATEWAY RESPONSIBILITIES · AVOIDING CHATTY SERVICE COMMUNICATION · HANDLING PARTIAL FAILURES



CORE RESPONSIBILITIES

- Request routing, authentication and authorization, request/response filtering
- Rate limiting and throttling, caching, security enforcement (TLS, WAF)
- Monitoring and logging, resilience (timeouts, retries, circuit breakers), API versioning



BEST PRACTICES

- Keep the gateway stateless, highly available, and free of business logic
- Monitor performance and set SLOs for the gateway itself



WHAT MAKES COMMUNICATION CHATTY

- Many small requests between services to complete one business operation
- High round-trip count increases latency, resource usage, and failure points



STRATEGIES TO REDUCE CHATTINESS

- Design coarse-grained APIs that return all data a client needs for a use case
- Use aggregation services, API composition, and caching to cut round-trips



STRATEGIES TO HANDLE PARTIAL FAILURES

- Timeouts to fail fast; retries with exponential backoff and jitter
- Circuit breakers to stop calling a failing service temporarily
- Fallbacks, bulkhead isolation, rate limiting, and graceful degradation

KEY TAKEAWAY Ten core responsibilities, from request routing to API versioning, all sit at the edge instead of being duplicated inside every microservice. • Coarse-grained APIs, aggregation services, and API composition each reduce round-trips that would otherwise multiply latency and failure points. • When Inventory Service is down, the rest of the system should degrade gracefully -- show cached data, hide a section -- rather than crash entirely.

DATA SERIALIZATION AND SCHEMA EVOLUTION · SELECTING THE RIGHT COMMUNICATION PATTERN



COMMON SERIALIZATION FORMATS

- JSON: human-readable, widely used for REST APIs and general communication
- Avro: compact, schema-based, strong typing -- common for Kafka event streaming
- Protocol Buffers: very compact and high performance for low-latency systems



SCHEMA EVOLUTION STRATEGIES

- Additive changes and optional fields are backward and forward compatible
- Deprecate before removing a field; version the schema for major changes

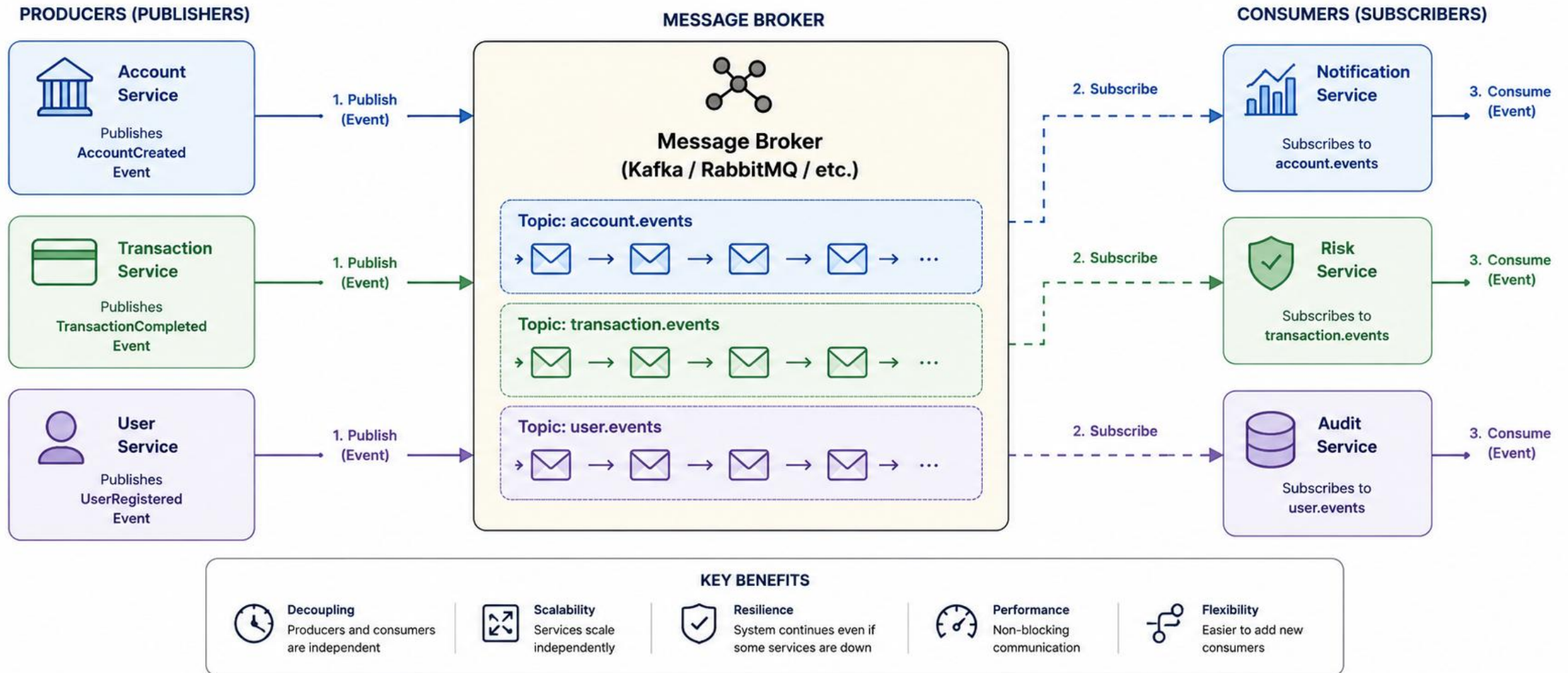


DECISION GUIDE

- Immediate response required? Yes -> synchronous. No -> asynchronous
- Low coupling important? -> asynchronous. Strong coupling acceptable? -> synchronous
- Strong/immediate consistency needed? -> synchronous. Eventual is fine? -> asynchronous

KEY TAKEAWAY Additive, optional changes stay backward and forward compatible; removing or renaming a field breaks consumers on both sides. • Immediate response needed, low coupling tolerance, or critical-path failure sensitivity each point toward synchronous or asynchronous differently.

ASYNCHRONOUS COMMUNICATION AND EVENT-DRIVEN ARCHITECTURE



INTRODUCTION TO ASYNCHRONOUS MESSAGING

A producer sends a message to a broker and moves on; the broker stores and delivers it independently, whenever the consumer is ready.



HOW ASYNCHRONOUS MESSAGING WORKS

1. Producer sends a message to a broker (queue or topic)
2. Broker stores and delivers the message independently
3. Consumer receives the message when it is ready to process it

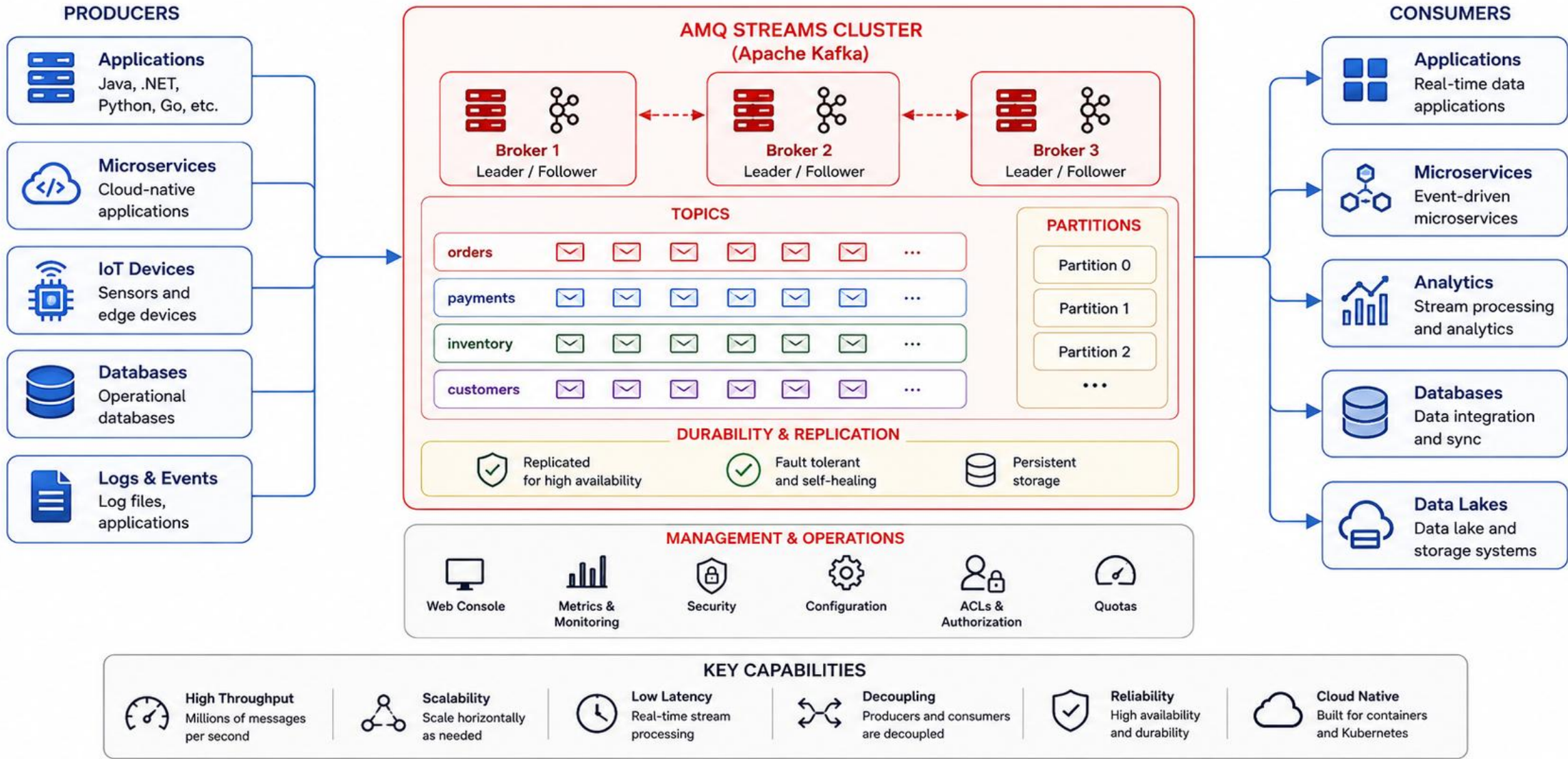


KEY CHARACTERISTICS

- Decoupling in time: sender and receiver don't need to be available simultaneously
- Improved scalability, resilience during outages, and eventual processing

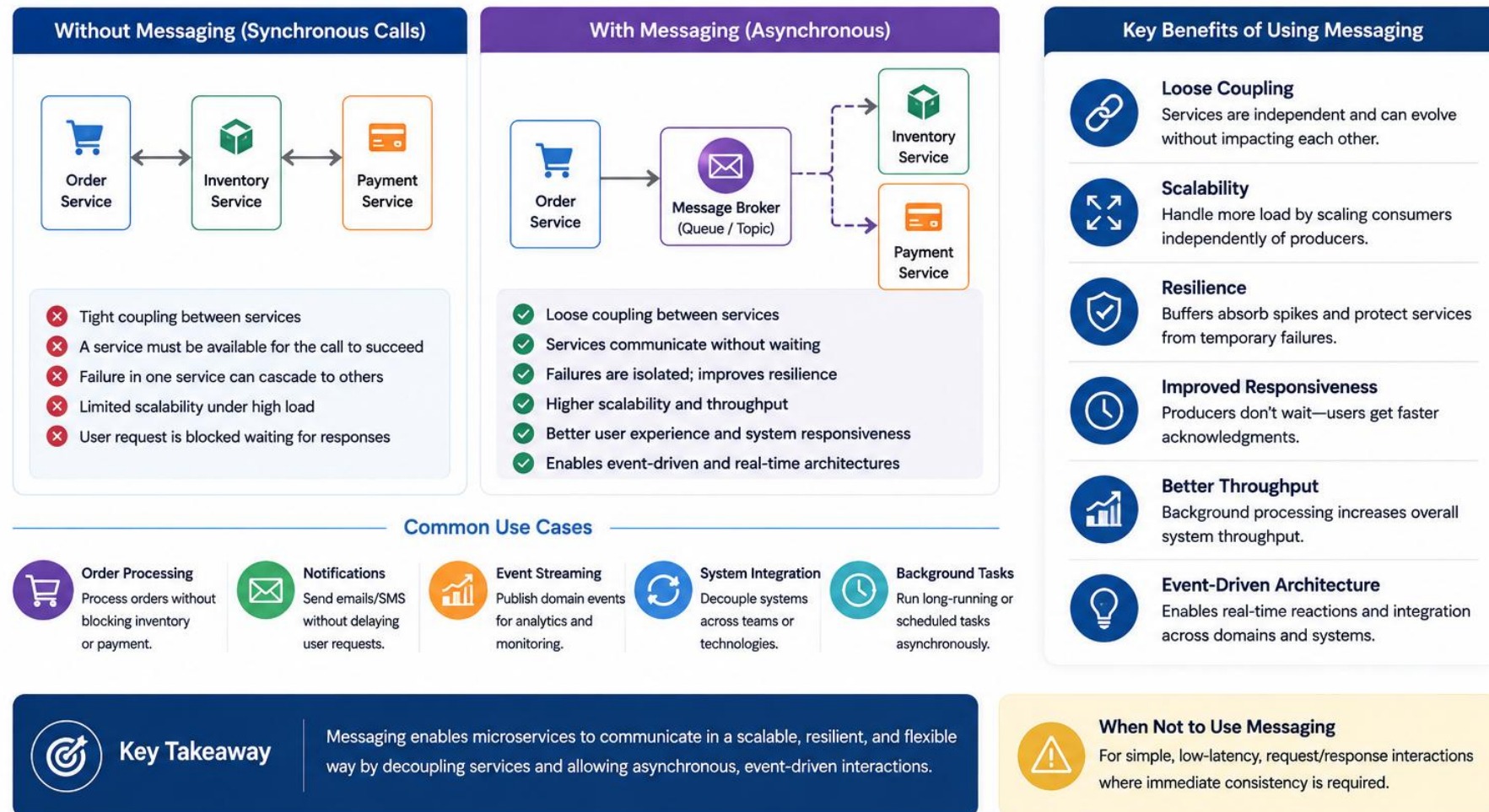
KEY TAKEAWAY A producer sends a message to a broker and moves on; the broker stores and delivers it independently, whenever the consumer is ready.

Red Hat AMQ Streams Overview



Why Use Messaging Between Microservices?

Messaging enables services to communicate asynchronously, improving decoupling, scalability, resilience, and responsiveness of distributed systems.



MESSAGE BROKERS AND THEIR RESPONSIBILITIES · QUEUES, TOPICS, AND SUBSCRIPTIONS



CORE RESPONSIBILITIES OF A BROKER

- Message ingestion, durable storage, routing and delivery to the correct consumers
- Delivery guarantees (at-most-once, at-least-once, exactly-once)
- Acknowledgment management, buffering/flow control, monitoring and management



QUEUES (POINT-TO-POINT)

- Each message is delivered to and consumed by one consumer
- Ideal for load balancing and task distribution (order processing, background jobs)

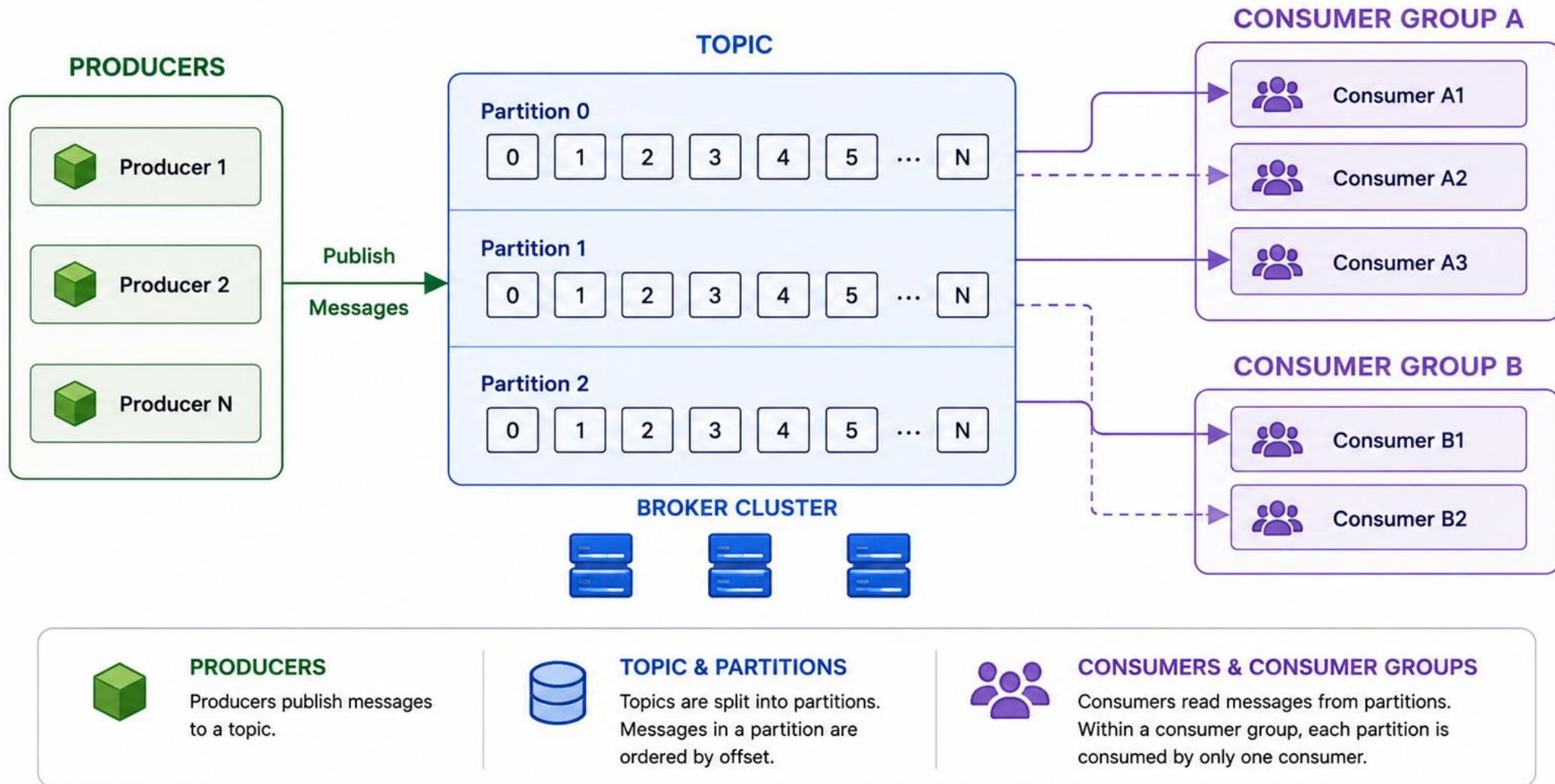


TOPICS (PUBLISH/SUBSCRIBE)

- Each message is delivered to all interested subscribers
- Ideal for event broadcasting and notifications; subscribers join or leave independently

KEY TAKEAWAY Without a broker, Account Service would need to know and call every consumer directly -- with one, it just publishes and moves on. • A queue removes a message once one consumer processes it; a topic keeps delivering the same message to every interested subscriber.

Producers, Consumers, and Consumer Groups



POINT-TO-POINT MESSAGING · PUBLISH-AND-SUBSCRIBE MESSAGING · COMMANDS, EVENTS, AND NOTIFICATIONS

HOW IT WORKS

1. Producer sends a message to a queue
2. Broker stores the message in the queue
3. A consumer retrieves and processes the message
4. The message is acknowledged and removed; unacknowledged messages can be redelivered

WHEN TO USE

- When multiple services need the same event or update
- When you want to decouple producers from consumers entirely

HOW IT WORKS

1. A publisher sends a message to a topic
2. The broker stores the message in the topic
3. All active subscribers receive a copy and process it independently

THREE MESSAGE TYPES

- Commands: request an action, sent to a single responsible service (point-to-point)
- Events: describe a fact that already happened, published to a topic, should be immutable
- Notifications: inform or alert, no action required, often consumed by external systems or UIs

KEY TAKEAWAY A message sits in the queue until acknowledged; if the consumer fails before acknowledging, the message becomes available again for another worker. • New subscribers can start receiving future messages without any change to the publisher -- the two sides stay completely decoupled. • A CreateOrderCommand asks for an action, an OrderCreatedEvent records that it happened, and an OrderShippedNotification simply informs interested parties.

EVENT-DRIVEN ARCHITECTURE · EVENT NOTIFICATION VS. EVENT-CARRIED STATE TRANSFER



EDA DESIGN PRINCIPLES

- Events over commands: share facts, not requests
- Immutability: events should never be changed or deleted
- Asynchronous first, idempotent consumers, single responsibility per event



EVENT NOTIFICATION (EN)

- A lightweight message that announces a fact -- contains only identifiers and metadata
- Subscriber must call back to the publishing service to get the data it needs



CHALLENGES AND CONSIDERATIONS

- Eventual consistency, added complexity, and the need for end-to-end monitoring
- Message ordering and duplicate delivery both require deliberate design

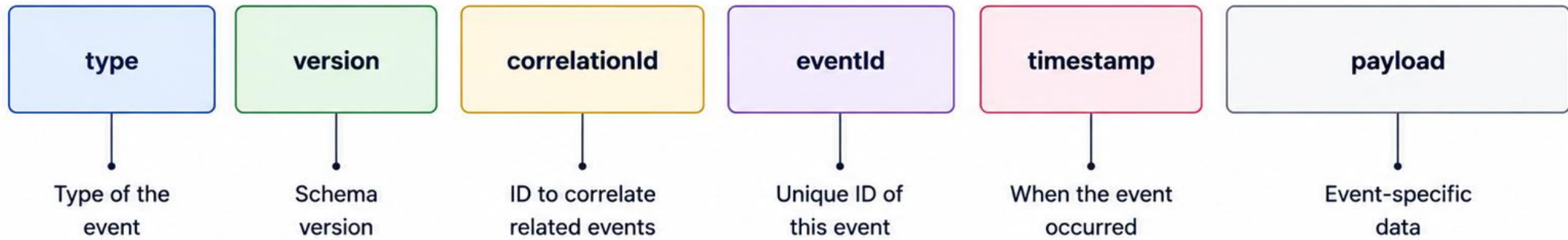


EVENT-CARRIED STATE TRANSFER (ECST)

- A self-contained message that includes the data subscribers need to process it
- Lower coupling and better resilience, at the cost of larger messages and data duplication

KEY TAKEAWAY Produce, publish, subscribe, react -- this four-step cycle is what lets services evolve, scale, and deploy independently of each other. • Event Notification keeps payloads small but adds a lookup call; Event-Carried State Transfer is self-contained but larger and harder to evolve.

Event Schema



Field	Description	Example	Notes
type	Type of the event	account.created	Event name in dot notation
version	Schema version of the event	1.0	Increment when schema changes in a non-backward-compatible way
correlationId	ID to correlate related events across services	550e8400-e29b-41d4-a716-446655440000	Same ID used by all events in the same business transaction/flow
eventId	Unique identifier of this event	c1f2d3e4-5678-4abc-9def-0123456789ab	Must be globally unique (UUID recommended)
timestamp	Timestamp when the event occurred (UTC)	2025-05-09T14:23:45.123Z	ISO 8601 format in UTC

MESSAGE PRODUCERS AND CONSUMERS · DELIVERY GUARANTEES

THE THREE GUARANTEES

GUARANTEE	DUPLICATES?	MESSAGE LOSS?	USE WHEN
At-Most-Once	Never	Possible	Occasional loss OK, duplicates not (metrics, logging)
At-Least-Once	Possible	No (normal conditions)	Every message must be delivered (orders, payments)
Exactly-Once	Never	Never (normal conditions)	Both loss and duplicates unacceptable (financial transactions)

PRODUCER KEY RESPONSIBILITIES

- Create messages with correct data and schema; choose the destination (topic or queue)
- Set headers, metadata, and partitioning keys; handle errors and retries when publishing

CONSUMER KEY RESPONSIBILITIES

- Subscribe to the right topic or queue; process messages correctly and efficiently
- Acknowledge or commit processing; handle failures and ensure reliability

KEY TAKEAWAY Produce, publish, consume, act -- the four-stage lifecycle, plus a pull or push delivery model, defines every producer/consumer relationship. • If in doubt, start with at-least-once and make consumers idempotent -- upgrade to exactly-once only when the use case truly requires it.

Published Event Vocabulary

Category	Event Name	Description	Key Attributes (Examples)	Published By	Consumed By
 Account Events	account.created	An account is created	accountId, customerId, accountType, currency, createdAt, correlationId	Account Service	Customer Service, Notification Service, Risk Service
	account.updated	An account is updated	accountId, changedBy, changes, updatedAt, correlationId	Account Service	Notification Service, Audit Service
	account.closed	An account is closed	accountId, closedBy, reason, closedAt, correlationId	Account Service	Customer Service, Notification Service, Audit Service, Risk Service
 Transaction Events	transaction.created	A transaction is created	transactionId, accountId, amount, currency, type, createdAt, correlationId	Transaction Service	Risk Service, Notification Service
	transaction.approved	A transaction is approved	transactionId, approvedBy, approvedAt, correlationId	Transaction Service	Notification Service, Audit Service
	transaction.settled	A transaction is settled	transactionId, settledAt, settlementRef, correlationId	Transaction Service	Account Service, Notification Service, Audit Service
	transaction.failed	A transaction fails	transactionId, reason, failedAt, correlationId	Transaction Service	Risk Service, Notification Service, Audit Service
 Customer Events	customer.registered	A customer is registered	customerId, name, email, phone, registeredAt, correlationId	Customer Service	Account Service, Notification Service, Risk Service
	customer.updated	A customer is updated	customerId, changedBy, changes, updatedAt, correlationId	Customer Service	Notification Service, Audit Service
	customer.deactivated	A customer is deactivated	customerId, deactivatedBy, reason, deactivatedAt, correlationId	Customer Service	Account Service, Notification Service, Risk Service
 Risk Events	risk.evaluated	A risk evaluation is completed	evaluationId, entityType, entityId, score, evaluatedAt, correlationId	Risk Service	Notification Service, Audit Service
	risk.flagged	A risk is flagged	flagId, entityType, entityId, reason, severity, flaggedAt, correlationId	Risk Service	Notification Service, Case Management Service
	risk.cleared	A risk flag is cleared	flagId, clearedBy, clearedAt, correlationId	Risk Service	Notification Service, Audit Service
 Notification Events	notification.sent	A notification is sent	notificationId, channel, to, subject, sentAt, correlationId	Notification Service	Audit Service
	notification.failed	A notification fails	notificationId, channel, to, reason, failedAt, correlationId	Notification Service	Audit Service

AT-MOST-ONCE DELIVERY · AT-LEAST-ONCE DELIVERY · EXACTLY-ONCE PROCESSING

- REALITY AND LIMITATIONS



WHAT IT MEANS

- A message is delivered zero or one time -- never more than once
- No retries, no acknowledgments; highest performance, lowest reliability



WHEN TO USE

- Metrics, telemetry, logging, and best-effort notifications
- Never use for financial transactions or critical updates



WHY DUPLICATES HAPPEN

- Ack lost after processing, consumer timeout, or consumer failure after processing but before ack
- In every case, the broker retries delivery, and the message is delivered again



CONSUMER RESPONSIBILITY: IDEMPOTENCY

- Idempotent operations, unique message IDs with a processed table, or upsert instead of insert
- Acknowledge only after successful processing, and use retries with exponential backoff



THE REALITY: LIMITATIONS

- Network failures are unavoidable; acks can be lost after processing but before committing
- "Exactly-once" is often scoped to the messaging system, not all downstream side effects
- External side effects (emails, payments) can't be undone if duplicates occur



WHAT WORKS IN PRACTICE

- Idempotent consumers, transactional outbox/inbox, and a deduplication store
- Monitor and reconcile with metrics, alerts, and periodic reconciliation

KEY TAKEAWAY No acknowledgment, no retry: the broker sends once and moves on, which is fine for telemetry but never for a financial transaction. • If an acknowledgment is lost, delayed, or never sent, the broker retries -- so the same message may be delivered and processed more than once. • Exactly-once guarantees inside a messaging system rarely extend to external side effects like emails or payments, which can't be rolled back.

IDEMPOTENT MESSAGE CONSUMERS

Because messages may be redelivered, idempotency is not optional in a message-driven system -- it's the baseline requirement for correctness.

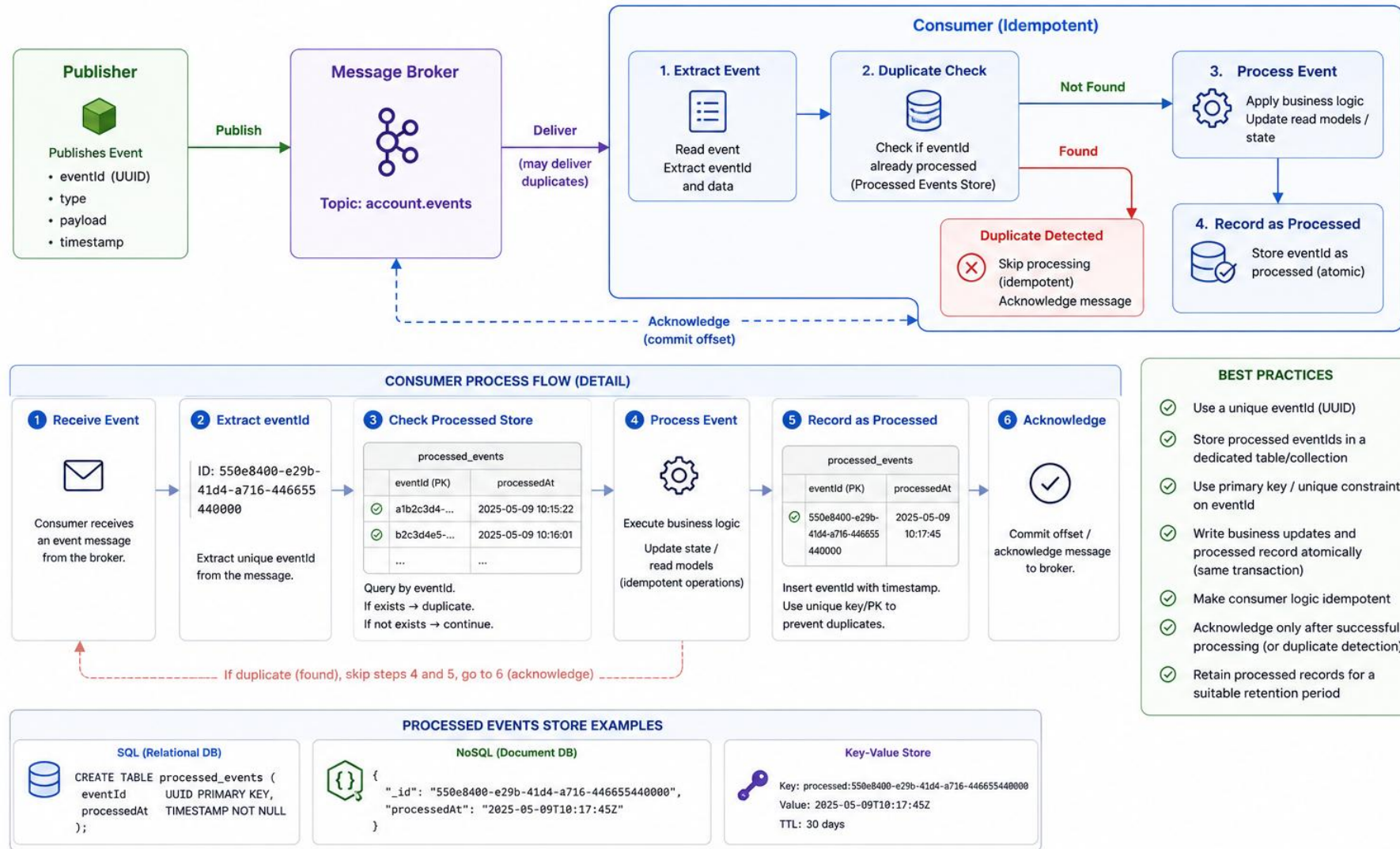


DUPLICATE HANDLING STRATEGIES

- Store processed event IDs and check before reprocessing
- Use unique business keys, upserts, or current-state checks before applying changes
- Use database constraints to enforce idempotency where appropriate

KEY TAKEAWAY Because messages may be redelivered, idempotency is not optional in a message-driven system -- it's the baseline requirement for correctness.

Idempotent Consumers and Duplicate Detection



MESSAGE ORDERING · MESSAGE PARTITIONING · CONSUMER GROUPS



ORDERING CONSIDERATIONS

- Kafka guarantees order within a partition, not across the whole topic
- RabbitMQ guarantees order within a single queue for a single consumer
- Design partition/routing keys deliberately when business order matters



HOW PARTITIONING WORKS

- A topic is divided into partitions, each an ordered, append-only log
- Messages with the same key always land in the same partition, preserving their order
- More partitions allow more parallel consumers, increasing overall throughput



HOW CONSUMER GROUPS WORK

- Consumers with the same group ID share a topic's partitions between them
- Each partition is assigned to exactly one consumer instance in the group at a time
- Different consumer groups each get their own full, independent copy of the event stream

KEY TAKEAWAY If a workflow depends on strict order (deposit before withdrawal), the partition key or routing key must keep those messages together. • Choosing a good partition key (like accountId) keeps all events for one entity in order while spreading load across the cluster. • Each partition is read by exactly one consumer within a group -- adding instances up to the partition count increases parallelism.

MESSAGE ACKNOWLEDGMENT AND REDELIVERY · DEAD-LETTER QUEUES · RETRY TOPICS AND BACKOFF STRATEGIES



ACKNOWLEDGMENT AND REDELIVERY FLOW

- Consumer receives a message and processes it
- On success, it sends an acknowledgment and the broker removes the message
- On failure or timeout with no ack, the broker redelivers the message



WHY DEAD-LETTER QUEUES MATTER

- Stores messages that could not be processed successfully after retries
- Prevents a single poison message from blocking the entire queue
- Allows teams to inspect, fix, and reprocess failed messages without losing them



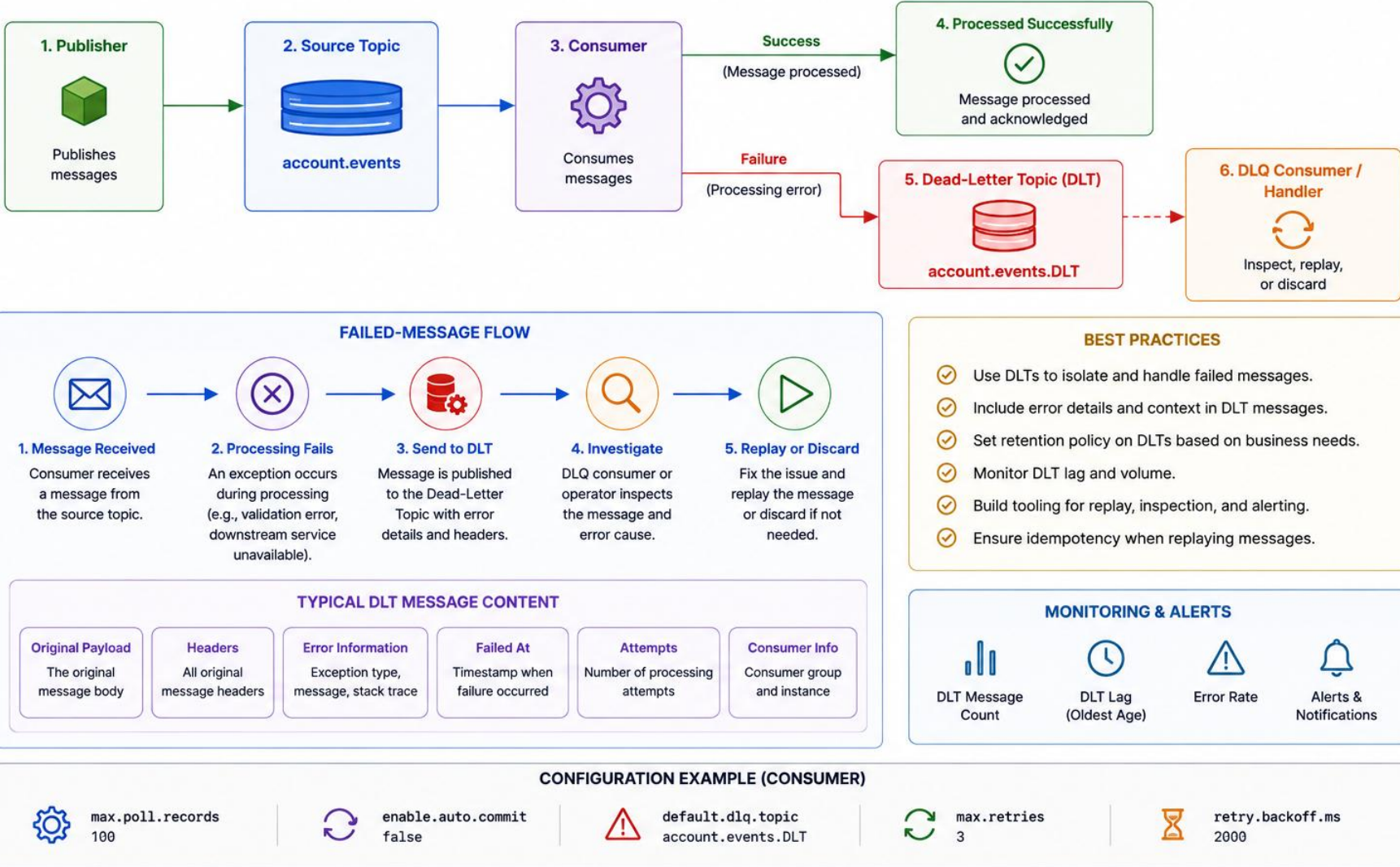
RETRY STRATEGY ELEMENTS

- Retry topics: separate topics/queues for staged retry attempts with increasing delay
- Exponential backoff: each retry waits longer than the last
- Jitter: randomizing delay slightly to avoid synchronized retry storms across consumers

KEY TAKEAWAY Manual acknowledgment after successful processing is safer than auto-acknowledgment, which can lose messages if the consumer crashes mid-processing.

- After a configured number of failed attempts, a message moves to the DLQ where operations teams can inspect and reprocess it manually.
- Exponential backoff with jitter spaces out retries so a temporary outage has time to recover instead of being hammered by retry storms.

Dead-Letter Topics and Failed-Message Handling



POISON MESSAGES

Without a retry limit and dead-letter routing, one bad message can block an entire queue and stall every message behind it.



HANDLING POISON MESSAGES

- A message that consistently fails processing regardless of retry attempts
- Cap retry attempts, then route to a dead-letter queue instead of retrying forever
- Alert operations and validate message schemas early to prevent poison messages

KEY TAKEAWAY Without a retry limit and dead-letter routing, one bad message can block an entire queue and stall every message behind it.

Integration Best Practices



1. Loose Coupling

- Communicate via well-defined contracts.
- Avoid tight dependencies between services.
- Use asynchronous events when possible.



2. Contract First

- Define API or event contracts upfront.
- Version contracts deliberately.
- Treat contracts as compatibility agreements.



3. Idempotency

- Design consumers to be idempotent.
- Use unique event IDs to detect duplicates.
- Safe to retry without side effects.



4. Reliability & Resilience

- Implement retries with exponential backoff.
- Use timeouts and circuit breakers.
- Handle failures gracefully and recover automatically.



5. Error Handling

- Handle errors explicitly and consistently.
- Use Dead-Letter Topics for failed messages.
- Capture and surface meaningful error details.



6. Data Ownership

- Each service owns its data.
- Access data via APIs or events, not direct DB access.
- Respect bounded contexts.



7. Observability

- Correlate requests/events using correlation IDs.
- Log structured, meaningful events.
- Monitor metrics and set actionable alerts.



8. Security

- Authenticate and authorize every integration.
- Encrypt data in transit (TLS) and at rest.
- Follow least privilege principles.



9. Versioning Strategy

- Version APIs and event schemas.
- Maintain backward compatibility.
- Deprecate and retire versions deliberately.



10. Performance

- Keep payloads small and efficient.
- Use pagination, filtering, and caching.
- Design for scalability and throughput.

Message Schema Design

A **message schema** defines the structure, data types, and validation rules for **messages** exchanged between producers and consumers. Well-designed schemas enable compatibility, evolution, and reliable processing.

EXAMPLE: ORDER_CREATED EVENT

```
JSON Schema (Example)
{
  "type": "object",
  "required": ["eventId", "eventType", "occurredAt", "order"],
  "properties": {
    "eventId": { "type": "string", "format": "uuid" },
    "eventType": { "type": "string", "enum": ["ORDER_CREATED"] },
    "occurredAt": { "type": "string", "format": "date-time" },
    "version": { "type": "integer", "minimum": 1 },
    "source": { "type": "string" },
    "order": {
      "orderId": { "type": "string" },
      "customerId": { "type": "string" },
      "orderDate": { "type": "string", "format": "date" },
      "totalAmount": { "type": "number" },
      "currency": { "type": "string", "pattern": "^[A-Z]{3}$" },
      "items": {
        "type": "array",
        "items": {
          "type": "object",
          "required": ["productId", "quantity", "unitPrice"],
          "properties": {
            "productId": { "type": "string" },
            "quantity": { "type": "integer", "minimum": 1 },
            "unitPrice": { "type": "number" }
          }
        }
      }
    }
  }
}
```

SCHEMA STRUCTURE

- Metadata**
Information about the event and its origin.
- Payload**
Business data relevant to the event.
- Identifiers**
Unique IDs for tracing, deduplication, and correlation.
- Version**
Schema version to manage compatibility.
- Extensibility**
Allow optional fields for future needs.

SCHEMA DESIGN CHECKLIST

Area	Key Considerations
Metadata & Naming	- Use descriptive event/message names. - Follow a consistent naming convention.
Fields & Types	- Choose appropriate data types. - Avoid ambiguous types (e.g., string for numbers).
Required vs. Optional	- Mark only truly essential fields as required. - Optional fields must have safe defaults.
Validation	- Define constraints (min/max, pattern, enum). - Validate at the producer and consumer.
Compatibility	- Backward compatibility: consumers can read new messages. - Forward compatibility: old messages can be read by new consumers.
Documentation	- Document fields, types, and meanings. - Maintain schema in a registry.
Governance	- Review changes and versions. - Deprecate fields carefully.

DOs

- Use clear, consistent field names.
- Keep messages small and focused.
- Include event time and source.
- Use strong typing and validation.
- Plan for schema evolution.
- Document and version schemas.
- Test schemas with real consumers.

DON'Ts

- Don't overload messages with unrelated data.
- Don't remove or rename fields without a plan.
- Don't change field types in incompatible ways.
- Don't rely on implicit contracts.
- Don't skip validation.

BENEFITS OF GOOD SCHEMA DESIGN

- Faster integration and onboarding**
- Fewer runtime errors and message rejects**
- Safe schema evolution over time**
- Better governance and observability**
- Stronger reliability and trust**

Remember:
Your schema is a contract. Design it with your consumers in mind, not just your current needs.

 COMPATIBILITY TYPES

CHANGE TYPE	BACKWARD	FORWARD
Add optional field	Compatible	Compatible
Add field with default	Compatible	Compatible
Remove field	Not compatible	Compatible
Rename or change type	Not compatible	Not compatible

 CORE KAFKA CONCEPTS

- Producer, topic, partition, consumer, consumer group, offset, broker, cluster
- A topic is a named stream of events; partitions allow Kafka to scale and parallelize
- An offset identifies a consumer's position in a partition, enabling replay

 CORE RABBITMQ CONCEPTS

- Producer, exchange, queue, binding, routing key, consumer, dead-letter exchange
- Exchange types: direct, topic, fanout, and headers exchanges each route differently
- The producer sends to an exchange; the exchange routes to one or more bound queues

KEY TAKEAWAY Adding an optional field keeps old consumers working and lets new consumers read old messages -- removing or renaming a field breaks both. • Producer -> Topic -> Partitions -> Consumers, with offsets tracking each consumer group's position -- the foundation of Kafka's event-streaming model. • Producer -> Exchange -> Binding/Routing Key -> Queue -> Consumer -- exchanges give RabbitMQ its flexible routing power.

Lab 2 Overview - Transaction Service



Purpose

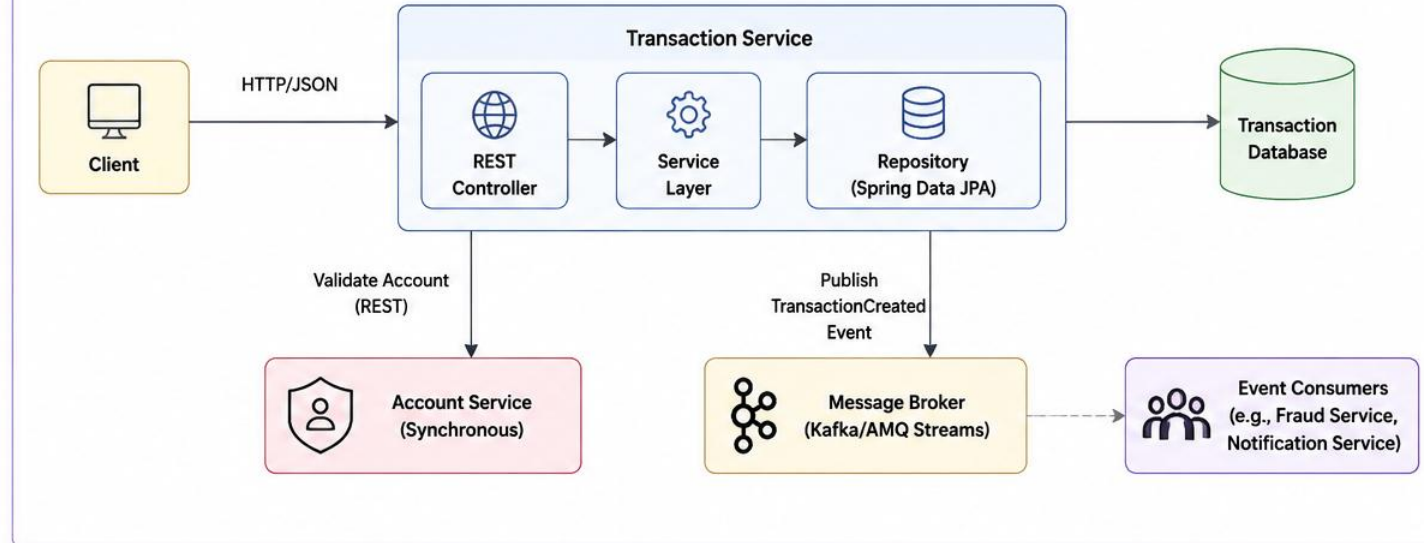
Build the Transaction Service to record and manage account transactions. Provide REST APIs and publish domain events.



Key Capabilities

- Create, read, and list transactions
- Validate account via Account Service (synchronous)
- Persist transactions (own database)
- Publish TransactionCreated event (asynchronous)
- Input validation and standardized errors
- Actuator, logging, and configuration

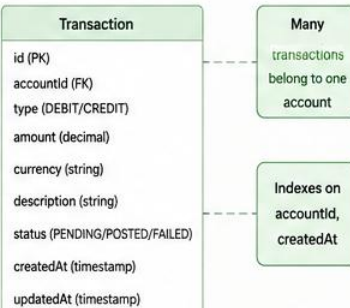
High-Level Architecture



Core API Endpoints

Method	Endpoint	Description
POST	/api/v1/transactions	Create a transaction
GET	/api/v1/transactions/{id}	Get transaction by ID
GET	/api/v1/accounts/{accountId}/transactions	List transactions by account
GET	/api/v1/transactions	List all transactions (paged)

Transaction Domain Model



Tech Stack

- Spring Boot 3
- Spring Data JPA
- PostgreSQL (Transaction DB)
- Spring Validation
- Kafka / AMQ Streams (Events)
- Spring Actuator

Deliverables

- ✓ Transaction Service Spring Boot application
- ✓ Database schema & JPA entities
- ✓ REST APIs implemented
- ✓ Account validation (sync call)
- ✓ Publish TransactionCreated event
- ✓ Unit tests
- ✓ README with run instructions

KAFKA VS. RABBITMQ · SPRING FOR APACHE KAFKA · SPRING AMQP AND RABBITMQ

KAFKA VS. RABBITMQ

AREA	RABBITMQ	KAFKA
Primary model	Message broker and queues	Event streaming platform
Message storage	Queue-oriented	Durable append-only log
Replay	Limited, pattern-dependent	Built-in through offsets
Common use	Task queues, routing	Event streams, analytics, high throughput

SPRING BOOT KAFKA INTEGRATION

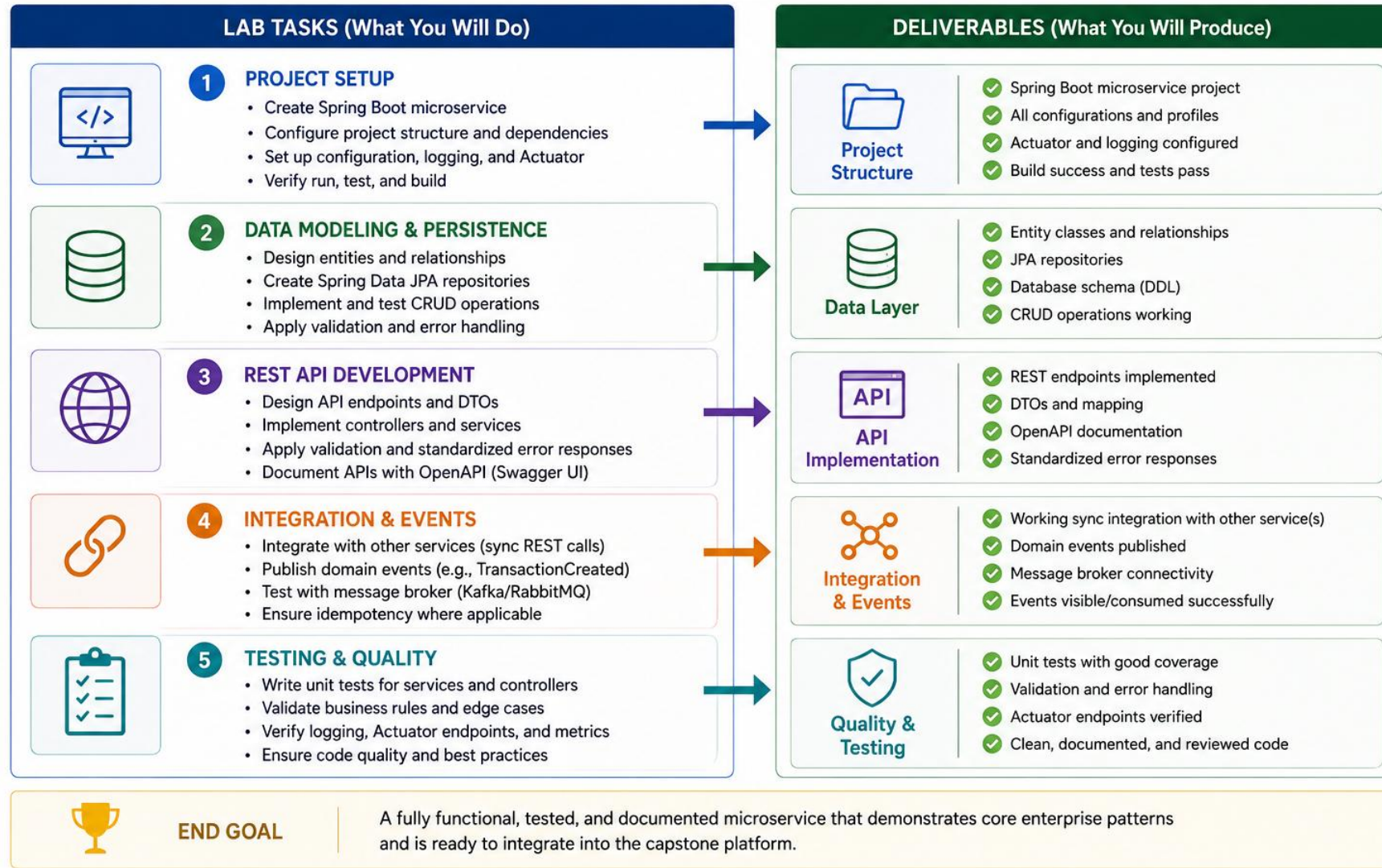
- `@KafkaListener(topics = "transaction-events", groupId = "fraud-service")`
- `KafkaTemplate` publishes messages; `@KafkaListener` methods consume them
- Each consumer group can process the same event stream independently

SPRING BOOT RABBITMQ INTEGRATION

- `RabbitTemplate` for publishing to an exchange with a routing key
- `@RabbitListener(queues = "notification.account-opened.queue")` for consuming
- Queue, exchange, and binding configuration ties the two together

KEY TAKEAWAY Choosing between them depends on whether the workload needs durable, replayable event history or flexible, routing-heavy task queues. • topics and groupId are all a listener needs to join a consumer group and start processing a topic's events. • A `RabbitTemplate.convertAndSend(exchange, routingKey, event)` call and a matching `@RabbitListener(queues=...)` method are all most services need.

Lab Tasks and Deliverables



TRANSACTIONAL MESSAGING CHALLENGES · THE TRANSACTIONAL OUTBOX PATTERN · CHANGE DATA CAPTURE



THE CORE PROBLEM

- A local database transaction and a broker publish cannot be committed atomically by default
- If the save succeeds but publishing fails, the event describing that save is lost



HOW THE OUTBOX PATTERN WORKS

1. Service writes the business change and an outbox row in one local transaction
2. A separate relay process reads unpublished outbox rows
3. The relay publishes each event to the broker and marks it as published



COMMON STRATEGIES

- Retry publishing, use the transactional outbox pattern, or mark events for later publishing



HOW CDC WORKS

- A CDC connector (e.g. Debezium) reads the database's transaction/replication log
- Each row-level change is converted into an event and published to a topic
- The application code doesn't need to explicitly publish events at all

KEY TAKEAWAY If the database commits but the broker publish fails (or vice versa), the two systems drift out of sync unless the workflow is designed for it.

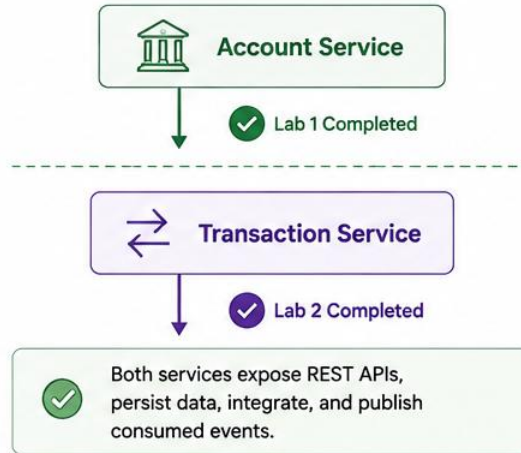
- A relay process reads unpublished rows from an outbox table and publishes them to the broker, guaranteeing the event is never silently lost.
- A tool like Debezium reads the database transaction log and turns every insert, update, and delete into a Kafka event automatically.

Day 2 Summary

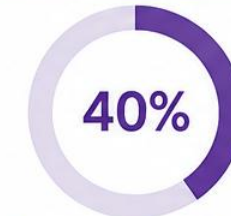
1. Core Concepts Covered

-  **Data Ownership & Isolation**
Database per service and why it matters.
-  **Consistency & Communication**
Eventual consistency, sagas, synchronous vs. async.
-  **Event-Driven Integration**
Domain events, brokers, Transactional Outbox pattern.
-  **Reliability & Resilience**
Idempotency, retries, error handling, and fault tolerance.

2. Hands-On Labs Completed



3. Capstone Progress



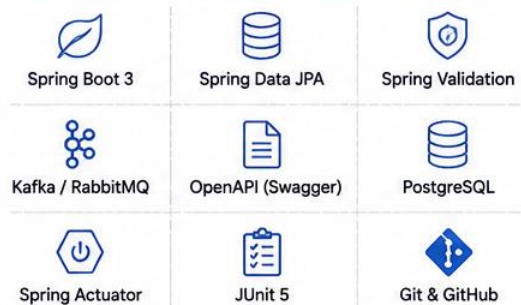
Capstone Completion

- ✓ Architecture foundation in place
- ✓ Account Service functional
- ✓ Transaction Service functional
- ✓ Event-driven integration implemented
- ✓ Ready for Risk & Notification Services

4. Key Patterns Applied

-  **Microservices Architecture**
Independent services with clear boundaries.
-  **Repository & Data Access**
Spring Data JPA for persistence.
-  **Event-Driven Communication**
Asynchronous messaging with domain events.
-  **Resilience Patterns**
Idempotency, retries, and robust error handling.

5. Tools & Technologies Used



6. What's Next

- Build Risk Service (Lab 3)
- Implement advanced rules & scoring
- Add Notification Service (Lab 4)
- Complete end-to-end saga flow
- Testing, hardening, and documentation



Key Takeaway

We built a solid microservices foundation with independent services, event-driven integration, and enterprise patterns—ready to extend our banking risk platform.



Journey Continues

Tomorrow: Risk Service, rules engine, and deeper saga orchestration.

EVENT SOURCING OVERVIEW · EVENT SOURCING VS. EVENT-DRIVEN ARCHITECTURE



HOW EVENT SOURCING WORKS

- Every state change is persisted as an immutable event in an append-only event store
- Current state is derived by replaying (or folding) all events for an entity
- Provides a complete audit trail and enables rebuilding state at any point in time



TWO DIFFERENT CONCERNS

- Event-driven architecture: an integration style where services communicate via events
- Event sourcing: a persistence style where a service's own state is derived from an event log
- The two are complementary but solve different problems and can be adopted independently

KEY TAKEAWAY Current state is derived by replaying every event from the beginning -- giving a full, auditable history of exactly how the state was reached. • A service can be event-driven without event sourcing, and event sourcing internally without publishing every event externally -- they're independent choices.

Knowledge Check



1 Why do we give each microservice its own database?

- A To reduce the number of databases
- B To ensure data independence and autonomy
- C To share data more easily between services
- D To avoid writing any SQL



2 What is the main difference between synchronous and asynchronous communication?

- A Synchronous is faster and asynchronous is slower
- B Synchronous waits for a response; asynchronous does not
- C Asynchronous uses HTTP; synchronous uses TCP
- D There is no difference in reliability



3 What is the purpose of domain events?

- A To update the database schema
- B To notify other services of something that happened
- C To replace REST APIs
- D To store logs permanently



4 What problem does the Transactional Outbox pattern solve?

- A Duplicate records in a database
- B Losing events if the service crashes after commit
- C Slow database performance
- D Large message sizes



5 What is idempotency in APIs?

- A The API runs only once
- B Multiple identical requests have the same effect as one
- C The API always returns a cached response
- D The API requires authentication



6 Why do we version APIs?

- A To make the URLs look cleaner
- B To manage changes without breaking existing clients
- C To improve database performance
- D To encrypt the API requests



7 What is the main use of Actuator in Spring Boot?

- A To build REST APIs
- B To monitor and manage the application
- C To connect to the database
- D To secure endpoints with OAuth



8 What is the best practice for API error responses?

- A Return plain text messages
- B Use standardized error structure with proper HTTP status
- C Return stack traces to help the client
- D Always return 200 OK with error in body



9 What is the benefit of event-driven integration?

- A Strong consistency across services
- B Loose coupling and better scalability
- C Eliminates the need for databases
- D Faster synchronous responses



10 Which pattern helps avoid distributed transactions?

- A Two-Phase Commit
- B Saga Pattern
- C Shared Database
- D Global Locking



Self Check

Review the topics from Day 1 & Day 2.
If unsure, revisit the concept before moving forward.



Key Takeaways

Data autonomy, smart communication, event-driven patterns, and resilience are the foundation of our platform.

MESSAGING OBSERVABILITY · MESSAGING SECURITY · EVENT-DRIVEN DESIGN ANTI-PATTERNS



WHAT TO MONITOR

- Correlation IDs propagated through every log, event, and consumer processing step
- Consumer lag: the gap between the latest available events and what's been processed
- Dead-letter queue depth, retry counts, and end-to-end message latency



SECURING MESSAGING INFRASTRUCTURE

- Require authentication and authorization for producers and consumers
- Encrypt messages in transit (TLS) and restrict network access to the broker
- Avoid including sensitive data (passwords, tokens, full account numbers) in payloads



COMMON ANTI-PATTERNS

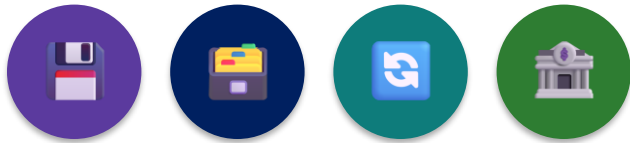
- Publishing events before the business transaction actually succeeds
- Vague event names such as DataChanged instead of specific business facts
- Ignoring duplicate event handling and assuming strict global ordering
- Building critical workflows on messaging without any monitoring

KEY TAKEAWAY Was a message delivered? Was it processed twice? Which consumer is falling behind? Observability answers questions REST tracing alone can't. • Authentication, authorization, encryption in transit, and careful payload design keep sensitive banking events from leaking through the broker. • Vague event names, ignored duplicates, and events published before the transaction succeeds are the most common ways event-driven design goes wrong.

LAB 2

Lab 2 — Transaction Service

A two-service workflow using REST validation and TransactionSubmitted events.



LAB 2 — TRANSACTION SERVICE — OVERVIEW

Day 2 lab: what you build and how it grows the capstone.



WHAT YOU WILL FINISH WITH

- Transaction Service on port 8082
- Its own database (transaction_db on host port 5434) — no shared tables with Account Service
- REST call to Account Service before money movement is accepted
- A TransactionSubmitted event on Kafka topic transactions.submitted
- An idempotent consumer that marks the row SUBMITTED
- Poison messages on transactions.submitted.DLT
- Passing unit tests and logs that use synthetic IDs only

KEY TAKEAWAY Lab 2 builds Transaction Service -- the working code behind today's concepts, and the next layer of the banking capstone.

LAB 2 — TRANSACTION SERVICE — BEFORE YOU START

Check these before the lab clock starts.

 KNOWLEDGE YOU NEED (FROM DAY 2)

Day 2 idea	How it appears in this lab
Database per service	Account data stays in account_db. Transaction rows live only in transaction_db.
Flyway	You version the transaction schema. Hibernate does not create tables.
Synchronous REST	Before insert, GET the account. Only ACTIVE accounts may transact.
Asynchronous Kafka	After insert, publish TransactionSubmitted. The consumer updates status later.
Event envelope	eventType, eventVersion, eventId, correlationId, occurredAt, payload.
Correlation ID	Header X-Correlation-Id is copied onto the event and into logs.
Idempotent consumer	Table processed_events keyed by event_id. A replay is a no-op.

 ENVIRONMENT BASICS

- Completed Lab 1 Account Service running on 8081 (your finished lab or labs/day-01/lab1/solution/account-service)
- Java 21
- Maven 3.9+
- Docker Desktop running (PostgreSQL and Kafka)

KEY TAKEAWAY Settle prerequisites and environment first -- most lab time is lost to setup, not to code.

LAB 2 — TRANSACTION SERVICE — LAB STEPS

9 steps, worked in order.

LAB STEPS

- Step 1 — Run and validate the starter
- Step 2 — Apply the Flyway migration
- Step 3 — Confirm entity, repository, and schema match
- Step 4 — Validate the account with REST
- Step 5 — Persist the transaction

LAB STEPS (CONTINUED)

- Step 6 — Publish TransactionSubmitted
- Step 7 — Consume, correlate, and ignore duplicates
- Step 8 — Exercise REST, then dead-letter routing
- Step 9 — Validation, health, docs, tests, log review

KEY TAKEAWAY Work the steps in order -- each one builds on the last.

LAB 2 — TRANSACTION SERVICE — SUCCESS CRITERIA

You are done when every box below is ticked.



SUCCESS CRITERIA

- Account Service (8081) and Transaction Service (8082) both healthy
- Flyway V1 applied on transaction_db only
- POST create on an ACTIVE account returns 201 RECEIVED
- GET later shows SUBMITTED
- Non-ACTIVE account returns 409



SUCCESS CRITERIA (CONTINUED)

- Correlation ID from the request appears in logs and on the event
- Duplicate eventId is ignored
- Poison Kafka message reaches transactions.submitted.DLT
- mvn test passes
- Logs stay synthetic and small

KEY TAKEAWAY Tick every criterion before moving on -- the next lab builds directly on this one.

DAY 2 COMPLETE

Data and Event-Driven Communication

Day 2 complete — A two-service workflow using REST validation and TransactionSubmitted events.

